

Grado Universitario en Ingeniería Informática
2025-2026

Trabajo Fin de Grado

“Evaluación de modelos de PLN para la detección de amenazas en ciberseguridad”

Hugo Becerra Fernández

Tutora

Isabel Segura Bedmar

Leganés, junio 2026



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

Este trabajo estudia la clasificación automática de frases extraídas de informes técnicos de malware. En concreto, se aborda la SubTarea 1 de SemEval-2018 Task 8 (SecureNLP), cuyo objetivo es decidir si una frase es relevante o no para describir el comportamiento del malware. Esta clasificación puede servir como paso previo para localizar información útil en informes de inteligencia de amenazas cibernéticas (CTI, *Cyber Threat Intelligence*).

Para ello, se comparan varias familias de modelos de Procesamiento del Lenguaje Natural (PLN): un modelo clásico SVM (*Support Vector Machine*) con representación TF-IDF (*Term Frequency-Inverse Document Frequency*), dos variantes BiLSTM (*Bidirectional Long Short-Term Memory*) con atención, modelos Transformer generales, modelos Transformer especializados en ciberseguridad y modelos evaluados mediante *prompting* sin fine-tuning. También se analiza el efecto del aumento de datos mediante *back-translation*, aplicando la ruta inglés-francés-inglés sobre frases de la clase relevante.

La evaluación se realiza sobre el conjunto de test oficial, formado por 618 frases, usando como métrica principal el F1-score de la clase relevante. El mejor resultado lo obtiene la replicación de la arquitectura BiLSTM del equipo Villani, con un F1 de 0,5541. Entre los modelos Transformer, SecureBERT 2.0 alcanza el mejor rendimiento, con un F1 de 0,5515. El aumento de datos no produjo una mejora apreciable en BERT, cuyo F1 pasó de 0,5164 a 0,5145 con back-translation. En cambio, los modelos evaluados mediante *prompting* quedan por debajo de los modelos ajustados.

Los resultados muestran que, en este conjunto de datos, el uso de modelos más grandes o recientes no garantiza necesariamente un mejor rendimiento. También indican que el preentrenamiento en textos de ciberseguridad puede ayudar, aunque su efecto depende del modelo concreto y del protocolo experimental seguido. Como aportación adicional, se publican el código fuente y el dataset preprocesado para facilitar la reproducción de los experimentos.

Finalmente, se muestra el marco normativo al que aplica este trabajo, así como los Objetivos de Desarrollo Sostenible (ODS) que se asocian al trabajo, una descripción de la metodología de desarrollo y planificación; y una representación del entorno socioeconómico junto con un presupuesto ajustado a los salarios y costes del sector en la actualidad.

Palabras clave: Clasificación de texto, ciberseguridad, inteligencia de amenazas, Transformers, BiLSTM, aumento de datos, SemEval.

DEDICATORIA

A Isabel, gracias por motivarme en primero y evitar que me rindiese. Si hoy estoy cerca de ser ingeniero, fue gracias a cruzarme contigo en Estructura de Datos y Algoritmos. Estoy encantado de que hayas sido mi tutora de TFG.

A mis amigos de Valdemoro, quienes me han visto crecer y no se han separado aún cogiendo caminos distintos.

A mis amigos de la universidad, en quienes he encontrado un lugar donde olvidarme de mis problemas. No puedo estar más orgulloso de haber pasado las dificultades de la carrera a vuestro lado.

A mi padre, por haberme enseñado que la informática es la profesión más interesante de todas, por motivarme a seguir estudiando y ayudarme a estudiar asignaturas que se me complicaban.

A Lupi, la mejor perrita del mundo. Me has acompañado desde mi infancia y vas a seguir conmigo en mi graduación, me hace muy feliz.

A mi madre, la persona más fuerte que conozco. Gracias por haberme aguantado en mis peores rachas y por no juzgarme nunca.

A mi ratoncita, por no rendirse nunca conmigo y ser mi mayor ilusión. Eres la mejor compañera de vida que una persona puede tener, gracias por ser mis ojos cuando no podía ver. Ojalá sigas siendo mi faro en todas las etapas de mi vida.

ÍNDICE GENERAL

1. INTRODUCCIÓN.	1
1.1. Motivación	1
1.2. Objetivos	1
1.2.1. Objetivo general	1
1.2.2. Objetivos específicos.	2
1.3. Descripción del entorno de trabajo	2
1.4. Estructura del documento	3
2. FUNDAMENTOS TEÓRICOS	4
2.1. Procesamiento del Lenguaje Natural	4
2.1.1. Representación de texto	4
2.2. Aprendizaje automático	5
2.2.1. Máquinas de vectores de soporte (SVM).	5
2.2.2. Naive Bayes	6
2.3. Aprendizaje profundo	7
2.3.1. Modelos Transformer	8
2.3.2. BERT y sus variantes	10
2.4. Modelos Transformer especializados en ciberseguridad	11
2.4.1. CyBERT	11
2.4.2. SecureBERT	12
2.4.3. CTI-BERT	12
2.4.4. SecureBERT 2.0	13
2.4.5. SecBERT	13
2.4.6. Comparativa de modelos	13
2.4.7. Modelos de lenguaje grandes y prompting.	14
2.5. Técnicas de aumento de datos.	15
2.6. Conceptos de seguridad	16
2.6.1. Malware.	16
2.6.2. Inteligencia de amenazas (CTI).	17

3. ESTADO DEL ARTE	18
3.1. SemEval-2018 Task 8: SecureNLP	18
4. ANÁLISIS DEL SISTEMA	23
4.1. Descripción general del sistema	24
4.2. Actores y alcance.	24
4.3. Requisitos funcionales	25
4.4. Requisitos no funcionales	28
5. DATASET Y PREPROCESAMIENTO	32
5.1. Distribución de clases	33
6. ENFOQUES	36
6.1. Diseño experimental	36
6.1.1. Tratamiento del desbalanceo	37
6.1.2. Entorno de experimentación.	37
6.2. Baseline: SVM con TF-IDF	37
6.2.1. Representación del texto.	38
6.2.2. Clasificador y optimización	38
6.3. BiLSTM con atención y GloVe	38
6.4. Transformers genéricos: BERT, RoBERTa y DeBERTa	40
6.5. Transformers de ciberseguridad.	40
6.6. Modelos de lenguaje grandes con <i>prompting</i>	41
6.6.1. Clasificación por inferencia textual (BART-MNLI).	42
6.6.2. Clasificación mediante instrucciones (FLAN-T5).	42
6.7. Aumento de datos: Traducción Circular (Back-translation)	43
6.8. Resumen de configuraciones	44
7. RESULTADOS Y DISCUSIÓN	45
7.1. Métricas utilizadas	45
7.2. Resultados	45
7.3. Análisis de errores	50
7.3.1. Matriz de confusión del mejor modelo.	50
7.3.2. Tipos de error más frecuentes.	51
7.3.3. Diferencias entre familias de modelos	52

7.4. Discusión	53
7.5. Limitaciones del estudio.	56
8. MARCO REGULADOR, IMPACTO SOCIOECONÓMICO Y SOSTENIBILIDAD	58
8.1. Marco regulador	58
8.1.1. Normativa europea de ciberseguridad	58
8.1.2. Reglamento General de Protección de Datos y LOPDGDD	58
8.1.3. Ley de Inteligencia Artificial de la Unión Europea (<i>AI Act</i>)	59
8.1.4. Propiedad intelectual y licencias	60
8.1.5. Consideraciones éticas y de uso dual.	60
8.2. Impacto socioeconómico	61
8.3. Contribución a los Objetivos de Desarrollo Sostenible	62
8.4. Metodología de desarrollo y planificación	64
8.4.1. Metodología incremental experimental	64
8.4.2. Planificación temporal	64
8.5. Presupuesto	65
8.5.1. Coste de personal.	65
8.5.2. Coste de infraestructura y software.	66
8.5.3. Presupuesto total	67
9. CONCLUSIONES Y TRABAJO FUTURO	68
BIBLIOGRAFÍA	71
DECLARACIÓN DE USO DE IAG	

ÍNDICE DE FIGURAS

2.1	Ilustración del hiperplano de máximo margen en una SVM lineal. Los puntos enmarcados son los vectores de soporte. Elaboración propia mediante scikit-learn [12].	6
2.2	Ilustración del mecanismo de auto-atención. Cada token genera tres vectores (Q, K, V) y las flechas representan los pesos de atención de “ <i>cifró</i> ” hacia el resto de tokens de la secuencia. Elaboración propia.	8
5.1	Distribución de clases en los conjuntos de entrenamiento (train+dev fusionados, 10.648 frases) y test (618 frases) del corpus SemEval-2018 Task 8 SubTask 1. El desequilibrio es notable en ambos subconjuntos, con la clase relevante representando el 23,4 % en entrenamiento y el 14,6 % en test. Elaboración propia.	33

ÍNDICE DE TABLAS

2.1	Comparativa de modelos de lenguaje para ciberseguridad	13
3.1	Resumen de enfoques por equipo en SecureNLP SubTask 1	22
4.1	Plantilla de requisitos	24
4.2	Requisito funcional número 01	25
4.3	Requisito funcional número 02	25
4.4	Requisito funcional número 03	26
4.5	Requisito funcional número 04	26
4.6	Requisito funcional número 05	27
4.7	Requisito funcional número 06	27
4.8	Requisito funcional número 07	28
4.9	Requisito no funcional número 01	28
4.10	Requisito no funcional número 02	29
4.11	Requisito no funcional número 03	29
4.12	Requisito no funcional número 04	30
4.13	Requisito no funcional número 05	30
4.14	Requisito no funcional número 06	31
5.1	Distribución del dataset por splits	33
6.1	Resumen de modelos evaluados	44
7.1	Resultados sobre el conjunto de test oficial de SemEval-2018 Task 8 Sub- Task 1. Precisión y recall corresponden a la clase relevante.	46
7.2	Matriz de confusión de la BiLSTM (replicación de Villani) sobre el con- junto de test	50
7.3	Ranking de modelos supervisados por F1 de la clase relevante	54
7.4	Comparativa entre los mejores modelos de este trabajo y los sistemas pre- sentados en la competición original	55
8.1	Planificación del trabajo por fases	65

8.2	Coste de personal (estimación)	66
8.3	Coste de infraestructura y software (estimación)	67
8.4	Presupuesto total (estimación académica)	67

1. INTRODUCCIÓN

1.1. Motivación

Las organizaciones públicas y privadas trabajan con un volumen creciente de informes técnicos de malware, boletines de vulnerabilidades y documentos de inteligencia de amenazas cibernéticas (CTI, *Cyber Threat Intelligence*) [1]. Una parte importante de estos documentos se publica en lenguaje natural y describe cómo se comporta un programa malicioso: qué archivos modifica, con qué servidores se comunica o qué técnicas emplea para persistir en un sistema. Extraer esa información de forma manual es costoso y exige tiempo de analistas con experiencia, un recurso escaso en un sector con alta demanda de perfiles especializados [2].

El Procesamiento del Lenguaje Natural (PLN) ofrece herramientas para automatizar una primera fase de este trabajo. Antes de extraer entidades o relaciones de un informe, conviene separar las frases que describen un comportamiento relevante del malware de las que solo aportan contexto, referencias bibliográficas o texto introductorio. Esta separación es precisamente el problema que plantea la SubTask 1 de SemEval-2018 Task 8 (SecureNLP) [3], un *benchmark* público construido sobre el corpus MalwareTextDB [4]. Resolver bien esta fase inicial permitiría que un analista dedicara más tiempo a las tareas que requieren criterio experto y menos a la lectura exhaustiva de documentos.

El interés de esta tarea va más allá de su aplicación directa. Se trata de un problema de clasificación de texto en un dominio técnico, con un dataset pequeño y un fuerte desbalanceo entre clases. Estas condiciones permiten comparar distintas familias de modelos de PLN en un escenario realista, donde no siempre es evidente que un modelo más grande o más reciente vaya a obtener mejores resultados. El objetivo de este trabajo no es construir un sistema productivo completo, sino evaluar de forma sistemática qué enfoques funcionan mejor en esta tarea concreta y por qué.

1.2. Objetivos

1.2.1. Objetivo general

El objetivo general de este trabajo es evaluar y comparar distintas familias de modelos de Procesamiento del Lenguaje Natural para la clasificación binaria de frases de informes de malware como relevantes o no relevantes, según la definición de la SubTask 1 de SemEval-2018 Task 8 [3]. La comparación se realiza sobre el conjunto de test oficial (618 frases, de las cuales 90 son relevantes) y toma como métrica principal el F1-score de la clase relevante, por tratarse de la clase minoritaria.

1.2.2. Objetivos específicos

A partir del objetivo general se definen los siguientes objetivos específicos:

1. Preparar y caracterizar el dataset SecureNLP/MalwareTextDB [4], incluyendo el preprocesamiento del texto, la definición de las particiones y el análisis del desbalanceo entre clases.
2. Implementar un baseline reproducible basado en un clasificador SVM (*Support Vector Machine*) con representación TF-IDF, que sirva de referencia para el resto de modelos.
3. Implementar y evaluar modelos recurrentes BiLSTM con atención, replicando la arquitectura del equipo ganador de la competición original [5].
4. Ajustar y evaluar Transformers genéricos (BERT, RoBERTa y DeBERTa) en la tarea de clasificación. [6], [7], [8]
5. Evaluar Transformers preentrenados en textos de ciberseguridad (CTI-BERT, SecureBERT 2.0 y SecBERT) y compararlos con los modelos genéricos para comprobar si el preentrenamiento de dominio aporta una ventaja observable. [9], [10], [11]
6. Estudiar el efecto de una técnica de aumento de datos por *back-translation* sobre la clase minoritaria.
7. Explorar la clasificación sin fine-tuning mediante *prompting*, con inferencia zero-shot y few-shot.
8. Analizar los errores de los modelos y situar los resultados obtenidos frente a los sistemas presentados en la competición original.
9. Publicar de forma abierta el código y el dataset preprocesado para favorecer la reproducibilidad del trabajo.

1.3. Descripción del entorno de trabajo

Toda la experimentación se llevó a cabo en la versión gratuita de Google Colab, con GPU (*Graphics Processing Unit*) T4 o, de forma ocasional, P100, y un límite de 16 GB de VRAM (*Video Random Access Memory*). Este entorno impone restricciones de memoria, un cupo diario de uso y desconexiones automáticas que condicionaron varias decisiones del trabajo, como el truncamiento de las secuencias en los Transformers o la imposibilidad de cargar modelos de lenguaje de gran tamaño. Estas limitaciones se detallan en el Capítulo 7.

El desarrollo se realizó en Python. Para los modelos clásicos se utilizó *scikit-learn* [12]; para los modelos recurrentes, TensorFlow/Keras; y para los modelos basados en

Transformers, PyTorch [13] junto con la librería *Transformers* de HuggingFace [14]. El dataset preprocesado se publicó en HuggingFace Hub¹ y el código en un repositorio público de GitHub².

1.4. Estructura del documento

El resto de la memoria se organiza como sigue. El Capítulo 2 presenta los fundamentos teóricos necesarios, desde las técnicas clásicas de representación de texto hasta los Transformers y los modelos de dominio. El Capítulo 3 revisa el estado del arte en torno a SemEval-2018 Task 8 y los sistemas presentados a la competición. El Capítulo 4 recoge el análisis del sistema, con sus requisitos y su alcance. El Capítulo 5 describe el dataset y el preprocesamiento aplicado, prestando atención a la distribución de clases. El Capítulo 6 detalla los métodos y modelos implementados, agrupados en cinco familias. El Capítulo 7 presenta y discute los resultados experimentales, junto con el análisis de errores y las limitaciones del estudio. El Capítulo 8 aborda el marco regulador, el impacto socioeconómico, la contribución a los Objetivos de Desarrollo Sostenible, la metodología de trabajo y el presupuesto. Por último, el Capítulo 9 reúne las conclusiones y propone líneas de trabajo futuro.

¹<https://huggingface.co/datasets/hugobecerra/DATASET3.0>

²<https://github.com/100495717/TFG-HUGO-BECERRA>

2. FUNDAMENTOS TEÓRICOS

El objetivo de este capítulo es presentar los fundamentos teóricos necesarios para comprender las técnicas y metodologías empleadas a lo largo de este trabajo.

Por ello, este capítulo cubrirá desde las técnicas más sencillas y clásicas de representación de texto hasta los Transformers, pasando por las redes neuronales recurrentes y técnicas de aumento de datos.

2.1. Procesamiento del Lenguaje Natural

El Procesamiento del Lenguaje Natural (PLN) es el área de la inteligencia artificial que permite a los ordenadores comprender y generar texto escrito por humanos [15]. Abarca tareas muy diversas: traducción automática, resumen de documentos, análisis de sentimientos, respuesta a preguntas, entre otras. En este trabajo nos centramos en clasificación de texto, concretamente en clasificación binaria de frases.

Para que un algoritmo pueda procesar texto, primero hay que convertirlo en una representación numérica. Existen varias formas de hacerlo, cada una con sus ventajas y limitaciones.

2.1.1. Representación de texto

La forma más simple es **Bag of Words**: se cuenta cuántas veces aparece cada palabra y se representa el documento como un vector de frecuencias. El problema es que trata igual palabras muy frecuentes pero poco informativas (como sustantivos muy comunes tipo “cosa” o verbos como “hacer”) que palabras que realmente aportan significado al texto

TF-IDF (Term Frequency-Inverse Document Frequency) mejora esto penalizando las palabras que aparecen en muchos documentos. Si una palabra aparece en muchos textos, se asume que es poco informativa; en cambio si una palabra aparece en pocos documentos puede ser más relevante. La fórmula combina la frecuencia del término en el documento (TF) con el logaritmo inverso de la frecuencia en el corpus (IDF):

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log \frac{|D|}{|\{d \in D : t \in d\}|} \quad (2.1)$$

donde t es el término, d el documento y D el corpus completo. Un valor alto de TF-IDF indica que el término es frecuente en el documento pero raro en el corpus, lo que sugiere que es relevante para caracterizar ese documento.

Tanto Bags of Words como TF-IDF generan vectores dispersos de dimensionalidad alta (una dimensión por cada palabra del vocabulario) y no capturan relaciones semánticas entre palabras.

Los **word embeddings** representan cada palabra como un vector denso de dimensionalidad reducida, típicamente entre 50 y 300 dimensiones (siendo 300 el valor más habitual en la práctica) [16]. La idea fundamental es que palabras con significados similares tengan vectores cercanos en el espacio.

Word2Vec [17] fue uno de los primeros métodos en popularizar esta idea. Propone dos arquitecturas:

- **CBOW** (Continuous Bag of Words): predice una palabra a partir de su contexto (las palabras que la rodean).
- **Skip-gram**: hace lo inverso, predice el contexto a partir de una palabra.

En ambos casos, el contexto se define mediante una **ventana deslizante** de tamaño w , que determina cuántas palabras a cada lado de la palabra objetivo se toman en cuenta durante el entrenamiento. [17]

GloVe (Global Vectors) [18] sigue una estrategia diferente. En lugar de entrenar una red neuronal, construye una matriz de co-ocurrencias que cuenta cuántas veces aparece cada par de palabras juntas en el corpus. A partir de esta matriz, se factoriza para obtener vectores densos que capturan las relaciones estadísticas globales entre palabras.

Los modelos de word embeddings tienen una limitación importante: asignan un único vector a cada palabra, independientemente del contexto. Por ejemplo, la palabra “banco” tendría la misma representación en “me senté en un banco del parque” que en “fui al banco a sacar dinero”. Los modelos Transformer, que veremos más adelante, generan **embeddings contextualizados** donde la representación de cada palabra depende de las palabras que la rodean.

2.2. Aprendizaje automático

El aprendizaje automático permite que los sistemas aprendan patrones a partir de datos [19]. En la tarea de clasificación de texto, los algoritmos clásicos más consolidados como líneas base (*baselines*) siguen siendo las Máquinas de Vectores de Soporte (SVM)[20] y Naive Bayes [21], debido a su probada robustez frente a espacios de características de alta dimensionalidad y corpus con escasez de datos [1], [22].

2.2.1. Máquinas de vectores de soporte (SVM)

SVM es un algoritmo de clasificación que busca el hiperplano que mejor separa las clases [23]. “Mejor” significa que maximiza el margen, es decir, la distancia entre el

hiperplano y los puntos más cercanos de cada clase.

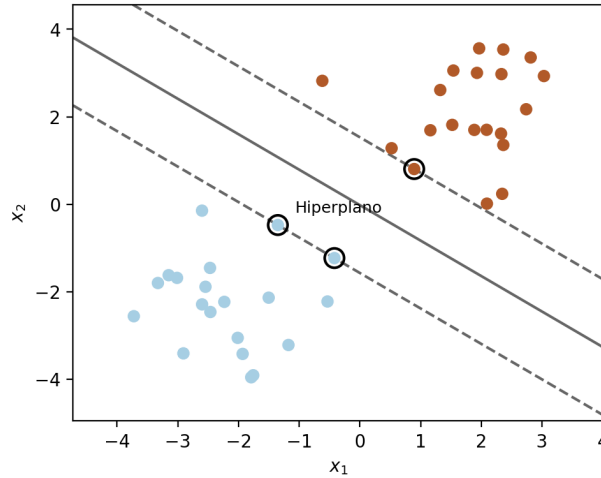


Fig. 2.1. Ilustración del hiperplano de máximo margen en una SVM lineal. Los puntos enmarcados son los vectores de soporte. Elaboración propia mediante scikit-learn [12].

Para datos que no son linealmente separables, SVM puede usar funciones kernel que proyectan los datos a un espacio de mayor dimensión donde sí lo sean. Sin embargo, en clasificación de texto con representación TF-IDF, donde los vectores tienen miles de dimensiones (una por cada palabra del vocabulario), el kernel lineal suele ser suficiente [24]. Esto se explica por el teorema de Cover [25]: en espacios de alta dimensionalidad, los datos tienen mayor probabilidad de ser linealmente separables. LinearSVC es una implementación eficiente de SVM lineal incluida en scikit-learn [12].

2.2.2. Naive Bayes

Naive Bayes es un clasificador probabilístico basado en el teorema de Bayes, propuesto para clasificación de texto por Maron y Kuhns [21]. Se llama “naive” (ingenuo) porque asume que las palabras son independientes entre sí dado la clase. Esta suposición es claramente falsa en texto real (por ejemplo, si aparece “inteligencia” es más probable que aparezca “artificial”), pero simplifica enormemente el cálculo y, sorprendentemente, funciona razonablemente bien en la práctica [26].

Esta suposición se basa en que modelar todas las dependencias entre palabras sería computacionalmente intratable. Si tenemos un vocabulario de 10.000 palabras, habría 2^{10000} posibles combinaciones. La independencia permite calcular la probabilidad de un documento como el producto de las probabilidades de cada palabra.

Para un documento d con palabras $w_1, w_2, \dots, w_n$ y una clase c , aplicando el teorema de Bayes:

$$P(c|d) = \frac{P(d|c) \cdot P(c)}{P(d)} \quad (2.2)$$

Como $P(d)$ es constante para todas las clases, basta comparar $P(d|c) \cdot P(c)$. Aplicando la suposición de independencia:

$$P(c|d) \propto P(c) \prod_{i=1}^n P(w_i|c) \quad (2.3)$$

donde $P(c)$ es la probabilidad a priori de la clase (estimada como la frecuencia de documentos de esa clase en el entrenamiento) y $P(w_i|c)$ es la probabilidad de la palabra w_i en documentos de clase c (estimada contando ocurrencias).

Por ejemplo, para clasificar el texto “el malware cifra los archivos” como relevante o no relevante, calcularíamos:

- $P(\text{relevante}) \cdot P(\text{malware}|\text{relevante}) \cdot P(\text{cifra}|\text{relevante}) \cdot \dots$
- $P(\text{no relevante}) \cdot P(\text{malware}|\text{no relevante}) \cdot P(\text{cifra}|\text{no relevante}) \cdot \dots$

Finalmente, la clase con mayor probabilidad es asignada.

2.3. Aprendizaje profundo

El aprendizaje profundo utiliza redes neuronales con capas múltiples para aprender representaciones de los datos [27]. En PLN, los primeros modelos de deep learning aplicados a clasificación de texto fueron las redes neuronales recurrentes [28] y las redes convolucionales [29].

Las **redes neuronales recurrentes** (RNN) procesan el texto palabra por palabra, manteniendo un estado interno que intenta capturar el contexto acumulado [28]. Sus principales variantes son **LSTM** (*Long Short-Term Memory*) [30] y **GRU** (*Gated Recurrent Unit*) [31], que incorporan mecanismos de puertas para controlar qué información se retiene o descarta a lo largo de la secuencia. En clasificación de texto se emplea habitualmente la versión bidireccional (**BiLSTM**), que procesa la secuencia en ambos sentidos para capturar contexto tanto anterior como posterior a cada palabra [32]. Sin embargo, tienen limitaciones importantes: el procesamiento secuencial impide la paralelización (lo que ralentiza el entrenamiento), y a pesar de las mejoras de LSTM, siguen teniendo dificultades para capturar dependencias muy largas.

Las **redes convolucionales** (CNN), originalmente diseñadas para imágenes, también se aplicaron a texto con buenos resultados [29]. Aplican filtros que detectan patrones locales (n-gramas) y pueden procesarse en paralelo. Sin embargo, tienen dificultades para capturar dependencias entre palabras distantes en el texto.

Estas limitaciones motivaron el desarrollo de la arquitectura Transformer.

2.3.1. Modelos Transformer

Arquitectura Transformer

En 2017, Vaswani et al. [33] propusieron la arquitectura Transformer. A diferencia de las RNN, que procesan el texto secuencialmente, los Transformers procesan toda la secuencia en paralelo. Esto ya fue conseguido por las CNN, pero seguían sin capturar dependencias entre palabras distantes. Los Transformers resuelven ambas limitaciones mediante dos mecanismos clave: la **auto-atención** (self-attention) y la **atención multi-cabeza** (multi-head attention) [34].

La **auto-atención** (self-attention) permite que cada palabra de la secuencia aprenda su representación con las demás, pero con diferentes pesos en base a su relevancia. Por ejemplo, en la frase “El malware que infectó el servidor cifró los archivos”, para entender “cifró” es importante relacionarlo con “malware” (el sujeto que realiza la acción), aunque estén separadas por varias palabras.

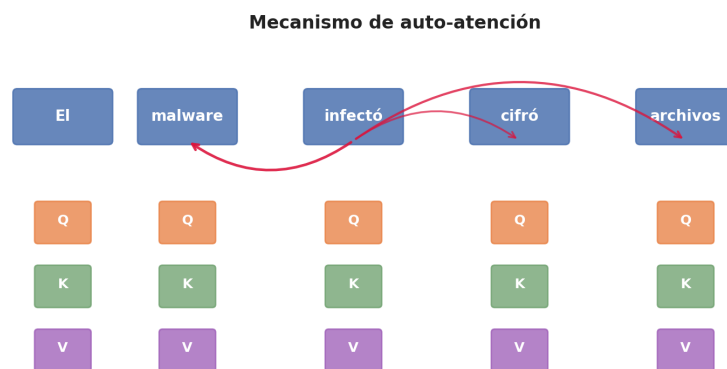


Fig. 2.2. Ilustración del mecanismo de auto-atención. Cada token genera tres vectores (Q, K, V) y las flechas representan los pesos de atención de “cifró” hacia el resto de tokens de la secuencia. Elaboración propia.

Para calcular la atención, cada token genera tres vectores mediante transformaciones lineales aprendidas:

- **Query (Q)**: representa qué información busca este token.
- **Key (K)**: representa qué información ofrece este token para ser encontrado.
- **Value (V)**: representa el contenido que este token aporta.

El proceso funciona así: para cada token, su vector Query se compara con los vectores Key de todos los tokens (mediante producto escalar). Esto genera una puntuación de relevancia para cada posición. Estas puntuaciones se normalizan con softmax para obtener pesos que suman 1. Finalmente, se calcula un promedio ponderado de los vectores Value

usando estos pesos. El resultado es una representación del token que incorpora información de toda la secuencia, ponderada por relevancia.

Matemáticamente:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.4)$$

La división por $\sqrt{d_k}$ (la raíz de la dimensión de los vectores Key) es un factor de escala que evita que los productos escalares sean demasiado grandes, lo que haría que softmax saturase.

La **atención multi-cabeza** ejecuta varios mecanismos de atención en paralelo, cada uno con sus propias matrices de proyección para Q, K y V. Cada “cabeza” puede aprender a capturar diferentes tipos de relaciones. Por ejemplo, en la frase anterior:

- Una cabeza podría aprender relaciones sintácticas (sujeto-verbo: “malware”-“cifró”).
- Otra podría capturar relaciones semánticas (“servidor”-“archivos” como elementos del sistema).
- Otra podría atender a modificadores (“que infectó” modifica a “malware”).

Las salidas de todas las cabezas se concatenan, obteniendo así la representación final de cada palabra. Esta representación ha sido construida con información procedente de toda la oración, ponderando simultáneamente distintos tipos de relaciones que cada cabeza ha aprendido a capturar.

Los modelos basados en Transformer se preentrenan con grandes cantidades de texto no etiquetado y luego se adaptan (*fine-tuning*) a tareas específicas. Existen diferentes familias según la arquitectura:

- **Solo encoder:** BERT [6], [35], RoBERTa [8], DeBERTa [7]. Se preentrenan mediante *Masked Language Modelling* (MLM) y son adecuados para tareas de comprensión (clasificación, extracción de información).
- **Solo decoder:** GPT [36], LLaMA [37]. Se preentrenan mediante *Causal Language Modelling* (CLM) y son adecuados para generación de texto.
- **Encoder-decoder:** T5 [38], BART [39]. Adecuados para tareas secuencia-a-secuencia (traducción, resumen).

Una consecuencia clave de este diseño es que los Transformers generan **embeddings contextualizados**: a diferencia de Word2Vec o GloVe, la representación de cada palabra varía según las palabras que la rodean en cada oración concreta.

Limitaciones de los Transformers: La auto-atención tiene complejidad cuadrática respecto a la longitud de la secuencia ($O(n^2)$), ya que cada token atiende a todos los demás [40]. Esto limita la longitud máxima de entrada en modelos como BERT (típicamente 512 tokens), aunque arquitecturas posteriores como Longformer [41] han extendido esta capacidad a secuencias mucho más largas mediante mecanismos de atención dispersa. Además, los Transformers requieren grandes cantidades de datos y recursos computacionales para ser preentrenados.

2.3.2. BERT y sus variantes

En este trabajo se emplean principalmente modelos de la familia BERT, ya que son arquitecturas de tipo *encoder-only* especialmente adecuadas para tareas de comprensión y clasificación de texto. A continuación se describen BERT y sus variantes más relevantes para este TFG.

BERT (Bidirectional Encoder Representations from Transformers) [6] utiliza solo la parte encoder del Transformer. Se preentrena con dos tareas:

- **Masked Language Model (MLM):** se enmascaran aleatoriamente el 15 % de los tokens y el modelo debe predecirlos a partir del contexto. Por ejemplo, dado “El [MASK] cifró los archivos”, el modelo debe predecir “malware”.
- **Next Sentence Prediction (NSP):** dado un par de frases, predecir si la segunda sigue a la primera en el texto original.

BERT-base tiene 110 millones de parámetros, 12 capas de Transformer, 768 dimensiones de embedding y 12 cabezas de atención. Se preentrenó con 3.300 millones de palabras de Wikipedia en inglés y libros.

Los modelos preentrenados pueden adaptarse a una tarea específica mediante *fine-tuning* (*fine-tuning*). En este proceso, el modelo se entrena de nuevo con datos etiquetados de la tarea objetivo, pero partiendo de las representaciones aprendidas durante el preentrenamiento. De esta forma, el conocimiento general adquirido por el modelo se ajusta al dominio y al problema concreto.

En tareas de clasificación de textos con BERT, lo habitual es añadir una capa lineal de clasificación sobre la representación del token especial [CLS], que se coloca al inicio de cada secuencia. Durante el fine-tuning, el modelo aprende a utilizar esa representación para predecir la etiqueta correspondiente a cada ejemplo.

RoBERTa (*Robustly Optimized BERT*) [8] mantiene la arquitectura general de BERT, pero modifica varios aspectos de su proceso de preentrenamiento. En primer lugar, elimina la tarea NSP (*Next Sentence Prediction*), utilizada en BERT para predecir si dos frases aparecen de forma consecutiva. Además, entrena con secuencias más largas, aplica enmascaramiento dinámico durante el entrenamiento, utiliza lotes de mayor tamaño

y amplía la cantidad de datos de preentrenamiento hasta aproximadamente 160 GB de texto.

Estas modificaciones buscaban mejorar la estabilidad y el rendimiento del modelo sin cambiar de forma sustancial su arquitectura. En la práctica, RoBERTa mostró mejores resultados que BERT en distintas tareas de referencia, lo que lo convirtió en una de las variantes más utilizadas dentro de los modelos Transformer de propósito general.

DeBERTa (Decoding-enhanced BERT with Disentangled Attention) [7] introduce dos mejoras técnicas: separa la codificación del contenido y la posición en el mecanismo de atención (disentangled attention), y añade información de posición absoluta justo antes de la capa de salida (enhanced mask decoder). Esto mejora especialmente en tareas que requieren entender relaciones posicionales.

2.4. Modelos Transformer especializados en ciberseguridad

Tras la aparición de BERT [6] y la generalización del esquema de preentrenamiento y fine-tuning (*fine-tuning*), se han desarrollado varios modelos Transformer específicos para ciberseguridad. La mayoría de ellos siguen una arquitectura de tipo *encoder-only*, similar a BERT, y están orientados principalmente a tareas de comprensión, clasificación o extracción de información.

A diferencia de modelos generales como BERT o RoBERTa, estos modelos se entrenan o reajustan con corpus del dominio de seguridad, como informes de amenazas, descripciones de vulnerabilidades CVE (*Common Vulnerabilities and Exposures*) y documentación técnica. El objetivo es que el modelo represente mejor el vocabulario propio del área, incluyendo nombres de malware, artefactos del sistema, técnicas de ataque, rutas, comandos o indicadores de compromiso.

Estos modelos no deben confundirse con los modelos de lenguaje grandes (LLM, *Large Language Models*) descritos en el capítulo anterior. Su escala suele ser más cercana a BERT-base y su finalidad principal no es generar texto, sino producir representaciones útiles para analizar y clasificar documentos de ciberseguridad.

2.4.1. CyBERT

CyBERT [42] fue uno de los primeros modelos BERT adaptados a ciberseguridad. Se desarrolló para clasificar declaraciones de características de seguridad en documentación de sistemas de control industrial (ICS).

- **Corpus de entrenamiento:** Documentación técnica de dispositivos ICS de múltiples fabricantes.
- **Método:** Fine-tuning de BERT-base con un proceso de re-etiquetado guiado por predicciones.

- **Arquitectura:** BERT-base con capas densas adicionales optimizadas mediante búsqueda de hiperparámetros.
- **Resultados:** Alcanzó un 94,4 % de accuracy en clasificación de declaraciones de seguridad, frente al 76 % del BERT original.

2.4.2. SecureBERT

SecureBERT [43] fue diseñado específicamente para tareas de inteligencia de amenazas cibernéticas (CTI). A diferencia de CyBERT, que solo realizó fine-tuning, SecureBERT implementó un preentrenamiento continuo con corpus de ciberseguridad.

- **Corpus de preentrenamiento:** Colección diversa que incluye informes de amenazas, artículos de Wikipedia sobre seguridad, tutoriales, CVEs y documentación técnica.
- **Método:** Preentrenamiento continuo desde RoBERTa con un tokenizador personalizado que incluye vocabulario específico de ciberseguridad.
- **Innovación:** Desarrollaron un método para ajustar los pesos preentrenados que mejora la eficiencia del aprendizaje en el nuevo dominio.
- **Resultados:** Superó a BERT y RoBERTa en tareas de MLM (Masked Language Model) con términos de ciberseguridad y en clasificación de textos de seguridad.

SecureBERT demostró que el preentrenamiento específico de dominio puede mejorar significativamente el rendimiento en tareas de ciberseguridad, especialmente cuando los textos contienen terminología técnica que no está bien representada en los corpus de preentrenamiento generales.

2.4.3. CTI-BERT

CTI-BERT [9] se desarrolló específicamente para el procesamiento de informes de inteligencia de amenazas. Su objetivo es mejorar tareas como la extracción de entidades de seguridad, la clasificación de tácticas y técnicas según MITRE ATT&CK, y la identificación de indicadores de compromiso.

- **Corpus:** Informes de amenazas, boletines de seguridad y documentación de vulnerabilidades.
- **Enfoque:** Preentrenamiento continuo con énfasis en la estructura y formato típico de los informes CTI.

2.4.4. SecureBERT 2.0

Recientemente, el equipo de Cisco AI presentó SecureBERT 2.0 [10], una actualización significativa que aborda limitaciones del modelo original:

- **Arquitectura:** Basado en ModernBERT, una actualización de la arquitectura BERT con mejor eficiencia para secuencias largas.
- **Corpus ampliado:** Incluye no solo texto de ciberseguridad sino también código fuente, lo que permite analizar vulnerabilidades en código.
- **Resultados:** Alcanza un F1-score de 0,945 en extracción de entidades de ciberseguridad, superando significativamente a SecureBERT (0,734) y CyBERT (0,351).

2.4.5. SecBERT

SecBERT [11] es otro modelo disponible públicamente que se entrenó específicamente con textos de ciberseguridad:

- **Corpus:** Textos de seguridad incluyendo el dataset de SemEval-2018 Task 8.
- **Método:** Preentrenamiento desde cero con vocabulario específico del dominio.
- **Disponibilidad:** Publicado en HuggingFace, lo que facilita su uso en investigación.

2.4.6. Comparativa de modelos

La Tabla 2.1 resume las características de los principales modelos de lenguaje para ciberseguridad:

Modelo	Base	Corpus	Año
CyBERT	BERT-base	Docs. ICS	2021
SecureBERT	RoBERTa	CTI, CVEs, tutoriales	2022
CTI-BERT	BERT	Informes CTI	2023
SecBERT	BERT (scratch)	Textos seguridad	2020
SecureBERT 2.0	ModernBERT	CTI + código	2025

TABLA 2.1. COMPARATIVA DE MODELOS DE LENGUAJE PARA CIBERSEGURIDAD

Estos modelos representan el estado del arte en PLN aplicado a ciberseguridad y serán evaluados en el Capítulo 7 para la tarea de clasificación de relevancia de SemEval-2018 Task 8.

2.4.7. Modelos de lenguaje grandes y prompting

Los modelos de lenguaje grandes (LLM, *Large Language Models*) son modelos basados en arquitecturas Transformer entrenados a una escala superior a la de modelos como BERT [44]. Mientras que BERT-base cuenta con 110 millones de parámetros, modelos posteriores han aumentado de forma notable su tamaño. GPT-2 [45], por ejemplo, alcanza 1.500 millones de parámetros; GPT-3 [46], 175.000 millones; y LLaMA 2 [47], hasta 70.000 millones.

Sin embargo, en los modelos más recientes la evolución no se explica solo por el número de parámetros. También influyen otros factores, como la forma de entrenamiento, el razonamiento, el uso de herramientas, la ventana de contexto y los mecanismos de alineamiento. En el caso de OpenAI, GPT-5 se presenta como un sistema que combina distintos modos de funcionamiento según la dificultad de la tarea [48]. Posteriormente, GPT-5.5 se describe como un modelo orientado a tareas complejas de programación, investigación, análisis de información y trabajo profesional [49].

A diferencia de modelos abiertos como LLaMA 2, estos modelos cerrados no publican de forma oficial su número exacto de parámetros. Por ello, en este trabajo no se comparan directamente por tamaño, sino por su forma de uso y por las restricciones prácticas del entorno experimental disponible.

Este aumento de escala ha permitido que los modelos de lenguaje grandes se utilicen en tareas que no siempre requieren un entrenamiento específico para cada problema. Entre ellas se encuentran la generación de texto, la respuesta a instrucciones, la clasificación mediante *prompting* o la resolución de tareas a partir de pocos ejemplos proporcionados en la entrada [44]. No obstante, estas capacidades dependen del modelo, del tamaño, del corpus de entrenamiento y de la forma en que se formula la instrucción, por lo que no deben interpretarse como propiedades garantizadas en todos los casos.

El preentrenamiento de muchos de estos modelos se basa en el *Causal Language Modelling* (CLM), o modelado causal del lenguaje. En este enfoque, el modelo aprende a predecir el siguiente token a partir de los tokens anteriores. Para ello se utilizan corpus de gran tamaño procedentes de fuentes diversas, como páginas web, libros, artículos o repositorios de texto. En el caso de GPT-3, el entrenamiento se realizó con cientos de miles de millones de tokens procedentes de varias fuentes textuales [46].

Una característica importante de los modelos de lenguaje grandes es que pueden adaptarse mediante *instruction tuning*. En este enfoque, el modelo no se entrena únicamente para resolver una tarea concreta, sino con ejemplos formulados como instrucciones en lenguaje natural. Esto mejora su capacidad para responder a tareas nuevas a partir de la propia descripción de la tarea, sin necesidad de realizar un fine-tuning específico en cada caso.

FLAN-T5 [50] es un ejemplo de este tipo de entrenamiento. Parte de la arquitectura T5, de tipo encoder-decoder [38], y se ajusta con instrucciones procedentes de múltiples

tareas. De esta forma, el modelo aprende a interpretar mejor órdenes escritas en lenguaje natural y a producir respuestas acordes con el formato solicitado.

Una propiedad llamativa de estos modelos es que tienen la capacidad de realizar tareas sin entrenamiento específico, mediante **prompting**: se describe la tarea en lenguaje natural y el modelo genera la respuesta [51].

En **zero-shot prompting**, solo se proporciona la descripción de la tarea:

Clasifica el siguiente texto como relevante o no relevante para ciberseguridad:

"El malware establece conexión con el servidor C2"

Respuesta:

En **few-shot prompting**, se incluyen ejemplos etiquetados junto con la instrucción para guiar al modelo.

Clasifica el siguiente texto como Relevante o No relevante para extracción de información sobre malware.

Texto: "El archivo contiene código malicioso" -> Relevante

Texto: "El documento describe la historia de la empresa" -> No relevante

Texto: "El troyano roba credenciales del sistema" -> Relevante

Texto: "La empresa fue fundada en 2010" -> No relevante

Texto: "El malware establece conexión con el servidor C2" ->

El último ejemplo se deja sin respuesta para que el modelo la complete a partir del patrón aprendido de los ejemplos anteriores.

2.5. Técnicas de aumento de datos

El **aumento de datos** (data augmentation) consiste en generar nuevos ejemplos de entrenamiento a partir de los existentes, modificándolos de forma que mantengan su significado y etiqueta [52]. Esta técnica tiene dos objetivos principales:

1. **Incrementar el tamaño del dataset:** Más datos de entrenamiento generalmente mejoran el rendimiento del modelo, especialmente en deep learning.
2. **Mejorar la generalización:** Las variaciones introducidas ayudan al modelo a ser más robusto y no sobreajustarse a las formas específicas de expresión del dataset original.

En texto, el aumento de datos puede aplicarse mediante distintas técnicas, como la sustitución de palabras por sinónimos, la inserción o eliminación aleatoria de términos,

el intercambio de posiciones, la *back-translation* y el parafraseo con modelos generativos [52]. Estas estrategias buscan generar nuevas variantes de las frases originales sin modificar de forma sustancial su significado.

Algunas de estas técnicas se han incorporado en herramientas específicas. **EDA** (*Easy Data Augmentation*) [53] reúne operaciones sencillas como sustitución, inserción, eliminación e intercambio de palabras. Por su parte, **NLPAug** [54] ofrece un marco más amplio, con estrategias de aumento a nivel de carácter, palabra y oración.

En este trabajo se presta especial atención a la *back-translation*, ya que fue una de las técnicas utilizadas en los experimentos. A continuación se describe su funcionamiento y cómo se aplicó en este proyecto.

Back-translation [55] consiste en traducir el texto a otro idioma (por ejemplo, de inglés a francés) y luego traducirlo de vuelta al idioma original. El resultado es un texto que expresa la misma información pero con distinta estructura semántica y elección de palabras.

Esta técnica funciona porque el lenguaje natural es inherentemente rico: una misma idea puede expresarse de múltiples formas válidas. Al pasar por otro idioma, el modelo de traducción elige una de las muchas formas posibles de expresar el contenido, generando variaciones legítimas.

Por ejemplo, partiendo de:

“The malware connects to a remote server to receive commands”

Tras back-translation inglés→español→inglés podríamos obtener:

“The malicious software establishes a connection with a remote server to obtain instructions”

Ambas frases son correctas, expresan lo mismo, y proporcionan variedad al dataset de entrenamiento.

2.6. Conceptos de seguridad

2.6.1. Malware

El **malware** (software malicioso) es cualquier programa diseñado con intenciones dañinas. Se clasifica según su comportamiento y objetivos:

- **Virus:** se replica insertando su código en otros programas.
- **Troyano:** se disfraza de software legítimo para engañar al usuario.

- **Ransomware:** cifra los archivos de la víctima y exige pago para recuperarlos.
- **Spyware:** recopila información del sistema sin conocimiento del usuario.
- **Rootkit:** se oculta en el sistema operativo para mantener acceso persistente.
- **Botnet:** red de equipos infectados controlados remotamente.

El análisis de malware estudia cómo funciona una muestra específica. Puede ser **estático** (examinar el código sin ejecutarlo) o **dinámico** (ejecutarlo en un entorno controlado y observar su comportamiento). Los informes de análisis describen: qué archivos crea o modifica, qué claves de registro altera, qué conexiones de red establece, qué técnicas de evasión utiliza, qué vulnerabilidades explota, etc.

2.6.2. Inteligencia de amenazas (CTI)

La **Inteligencia de Amenazas Cibernéticas** (Cyber Threat Intelligence, CTI) es el proceso de recopilar, procesar y analizar información sobre amenazas para tomar decisiones de seguridad informadas [2].

Un componente clave de CTI es la extracción de información estructurada a partir de fuentes no estructuradas (informes, artículos, foros). Esta información incluye:

- **Indicadores de compromiso (IoC):** hashes de archivos maliciosos, direcciones IP de servidores de comando y control (C2), dominios maliciosos, URLs de descarga de malware.
- **Tácticas, técnicas y procedimientos (TTP):** descripciones del comportamiento del atacante, típicamente categorizadas según frameworks como MITRE ATT&CK.
- **Información de actores:** identificación de grupos de amenazas, sus motivaciones y objetivos típicos.

El procesamiento manual de informes de amenazas es costoso y lento: un analista puede tardar horas en extraer la información relevante de un informe técnico extenso. Automatizar esta extracción mediante PLN permite procesar grandes volúmenes de documentación de forma eficiente [1].

3. ESTADO DEL ARTE

En este capítulo se revisan los trabajos previos relacionados con técnicas de Procesamiento del Lenguaje Natural aplicado a Ciberseguridad. En concreto, analizaremos la competición SemEval-2018 Task (SecureNLP) [3], que conforma el marco de evaluación de este trabajo, así como los enfoques que propusieron los participantes.

3.1. SemEval-2018 Task 8: SecureNLP

SemEval (*Semantic Evaluation*) es una serie de competiciones internacionales organizadas anualmente por la comunidad de PLN con el objetivo de evaluar y comparar sistemas de análisis semántico del lenguaje. Cada edición propone una o varias tareas compartidas (*shared tasks*) sobre problemas concretos, y los equipos participantes envían sus sistemas para ser evaluados con los mismos datos y métricas. Este formato de competición ha impulsado el avance del estado del arte en tareas como el análisis de sentimientos, la resolución de correferencia o la extracción de información, al permitir comparaciones directas entre enfoques muy distintos [3].

La competición SemEval-2018 Task 8, cuyo nombre es SecureNLP (Semantic Extraction from CybersecUrity REports using Natural Language Processing), fue organizada por Phandi et al. como la primera tarea compartida de SemEval centrada en ciberseguridad. El objetivo era el desarrollo de sistemas que fuesen capaces de extraer información semántica de informes técnicos sobre malware.

9 equipos participaron en la competición con un total de 18 envíos, reflejando el creciente interés de la comunidad de PLN en el dominio de ciberseguridad.

La competición se basó en el corpus MalwareTextDB [4], el primer dataset anotado de gran escala para extraer información de textos de malware. Este corpus sentó las bases para esta competición.

El dataset consiste en 39 informes de APT (Advanced Persistent Threat) anotados de forma manual, con un total de 6.819 frases.

Estos informes provienen de diversas fuentes públicas de inteligencia de amenazas, y describen campañas reales de malware. El contenido de ellas es heterogéneo: algunos informes son muy técnicos y detallan indicadores de compromiso específicos, mientras que otros incluyen secciones de contexto, análisis de motivaciones de los atacantes o recomendaciones de mitigación. Esta variabilidad es lo que hace complicado la tarea de distinguir qué frases son relevantes para la extracción de información de malware.

El esquema de anotación se basa en el vocabulario MAEC (Malware Attribute Enumeration and Characterization) [56], un estándar desarrollado por MITRE para describir

características de malware de forma estructurada.

Las anotaciones incluyen:

- **Relevancia:** si una frase contiene información relevante sobre el comportamiento del malware.
- **Entidades:** tokens que representan objetos involucrados (archivos, direcciones IP, claves de registro, etc.).
- **Acciones:** verbos que describen comportamientos del malware (conectar, cifrar, descargar, etc.).
- **Modificadores:** información adicional sobre entidades o acciones.

La competición se estructuró en cuatro subtareas incrementales:

1. **SubTask 1 - Clasificación de relevancia:** Tarea de clasificación binaria para determinar si una frase es relevante o irrelevante para la extracción de información sobre malware. Una frase se considera relevante si describe acciones, comportamientos o características del malware analizado.
2. **SubTask 2 - Etiquetado de tokens:** Tarea de etiquetado de secuencias para identificar tokens que representan Acciones, Entidades o Modificadores dentro de las frases relevantes.
3. **SubTask 3 - Predicción de relaciones:** Dada una frase relevante y sus tokens etiquetados, predecir las relaciones entre Acciones, Entidades y Modificadores.
4. **SubTask 4 - Predicción de atributos:** Clasificar las Acciones identificadas según una taxonomía de 454 tipos de comportamiento de malware definidos en MAEC.

Este trabajo se centra en la SubTask 1, que es fundamental porque las subtareas posteriores dependen de identificar correctamente las frases relevantes.

Los organizadores de la competición proporcionaron un sistema baseline para la SubTask 1 consistente en un clasificador SVM con características TF-IDF de bigramas, que obtuvo un F1-score de 0,5517 para la clase relevante. Este resultado ya era competitivo y solo fue superado por el mejor sistema participante, el cual veremos en Sistemas Participantes.

A continuación, se describen los sistemas que obtuvieron los mejores resultados en la tarea.

El equipo Villani [5], de IBM Research, obtuvo el mejor resultado en la SubTask 1 con un F1-score de 0.5714 para la clase relevante. Su enfoque se basó en:

- Una arquitectura BiLSTM (Bidirectional Long Short-Term Memory) con mecanismo de atención para capturar dependencias a largo plazo en las frases.
- Embeddings preentrenados GloVe [18] de 300 dimensiones como representación de entrada.
- Técnicas de sobremuestreo y ajuste de pesos en la función de pérdida para mitigar el desbalanceo de clases.
- Estudios de ablación comparando diferentes tipos de embeddings (Word2Vec, GloVe, FastText) y niveles de representación (palabra, carácter).

El análisis de atención fue clave para poder identificar qué palabras contribuían más a la clasificación, proporcionando así un grado de interpretabilidad al modelo.

El equipo Flytxt_NTNU [57] obtuvo el segundo mejor resultado con un F1-score de 0.5517, igualando prácticamente al baseline. Su enfoque combinó:

- Un clasificador SVM lineal con representación TF-IDF de bigramas como sistema principal.
- Un clasificador Naive Bayes como sistema complementario.
- Características adicionales basadas en técnicas de clustering de palabras para capturar información semántica.

Este resultado demuestra la competitividad de los métodos clásicos de aprendizaje automático frente a las arquitecturas de deep learning cuando se emplea un buen preprocesamiento y selección de características.

El equipo HCCL [58] propuso un sistema end-to-end basado en BiLSTM-CNN-CRF [59]:

- Combinaron representaciones a nivel de palabra (embeddings preentrenados) y a nivel de carácter (CNN sobre caracteres).
- La capa BiLSTM capturaba contexto bidireccional.
- Para la SubTask 1, utilizaron una capa de clasificación sobre la representación de la frase.

Obtuvieron el cuarto puesto en la SubTask 1, pero destacaron especialmente en la SubTask 2 bajo el esquema de evaluación relajado.

El equipo TeamDL [60] utilizó una CNN (Convolutional Neural Network) para clasificación de texto:

- Aplicaron la arquitectura de [29] adaptada al dominio de ciberseguridad.
- Experimentaron con diferentes embeddings: Word2Vec genérico, GloVe, y embeddings entrenados específicamente en textos de ciberseguridad.
- Utilizaron múltiples tamaños de filtro convolucional para capturar n-gramas de diferentes longitudes.

El equipo UMBC [61] fue el único que participó en las cuatro subtareas. Para la SubTask 1:

- Utilizaron embeddings entrenados específicamente con el corpus APTnotes, una colección de informes de amenazas.
- Emplearon un clasificador SVM con representación TF-IDF combinada con los embeddings de dominio.
- Experimentaron con diferentes estrategias de combinación de características.

El equipo Digital Operatives [62] adoptó un enfoque basado en características sintácticas:

- Utilizaron análisis de dependencias sintácticas como características adicionales.
- Emplearon un clasificador SVM con representación TF-IDF.
- Su análisis de errores reveló que la tarea es muy difícil, requiriendo en algunos casos conocimiento profundo del dominio y contexto extenso para clasificar correctamente las frases.

La Tabla 3.1 resume los enfoques y resultados de los principales equipos en la SubTask 1:

Equipo	Enfoque	F1 (Relevante)
Villani	BiLSTM + Atención + GloVe + sobre-muestreo	0,5714
Flytxt_NTNU	SVM + TF-IDF bigramas + clustering	0,5517
<i>Baseline</i>	<i>SVM + TF-IDF bigramas (organizados)</i>	<i>0,5517</i>
NLP Foundation	BiLSTM + embeddings preentrenados	0,5130
HCCL	BiLSTM-CNN-CRF end-to-end	0,4904
TeamDL	CNN para clasificación de texto	0,4871
UMBC	SVM + embeddings de dominio (APTnotes)	0,4561
Digital Operatives	SVM + TF-IDF + dependencias sintácticas	0,4213

TABLA 3.1. RESUMEN DE ENFOQUES POR EQUIPO EN SECURENLP SUBTASK 1

Los resultados muestran que la tarea es compleja: incluso el mejor sistema apenas supera un F1-score de 0,57 para la clase relevante. Solo el equipo Villani logró superar el baseline de los organizadores.

Los enfoques basados en deep learning (BiLSTM, CNN) obtuvieron resultados mixtos, sin una ventaja clara sobre los métodos clásicos. Esto sugiere que, para este dataset de tamaño moderado y con alto desbalanceo, la arquitectura del modelo es menos determinante que el tratamiento del desbalanceo de clases y la calidad de las representaciones.

4. ANÁLISIS DEL SISTEMA

Mediante la definición de requisitos se concreta qué debe cumplir el sistema planteado en este capítulo.

Este trabajo tiene una orientación principalmente investigadora. No se desarrolla un sistema completo en producción, sino que se evalúan distintos modelos que podrían integrarse en un sistema de clasificación automática de frases extraídas de informes técnicos de malware. La finalidad de ese sistema sería determinar si una frase es relevante o no para extraer información sobre el comportamiento de un programa malicioso.

Por tanto, los requisitos se formulan tomando como referencia un sistema hipotético de análisis de inteligencia de amenazas cibernéticas (CTI, *Cyber Threat Intelligence*). En este escenario, los modelos descritos en el Capítulo 6 se integrarían como componente de clasificación dentro de una herramienta utilizada por un analista de ciberseguridad, o por un proceso automatizado, para revisar informes de malware y localizar información accionable.

Se distinguen dos tipos de requisitos: funcionales y no funcionales. Los requisitos funcionales describen qué debe hacer el sistema, es decir, las acciones que debe permitir y los comportamientos que debe ejecutar. Los requisitos no funcionales describen cómo debe funcionar, incluyendo aspectos como rendimiento, escalabilidad, mantenibilidad, seguridad y reproducibilidad. En los escenarios definidos en este capítulo, el actor principal es un **analista de ciberseguridad** que utiliza el sistema para procesar informes de amenazas.

Cada requisito se describe en una tabla independiente basada en la plantilla de la Tabla 4.1, que incluye los siguientes campos:

- **Identificador:** código único que distingue cada requisito. El formato es RX-N, donde X toma el valor F para los requisitos funcionales y NF para los no funcionales, y N es el número secuencial comenzando en 01.
- **Título:** frase breve que resume el objetivo del requisito.
- **Prioridad:** nivel de importancia del requisito. Puede tomar tres valores: Alta, Media o Baja.
- **Descripción:** explicación detallada del propósito y finalidad del requisito.
- **Escenario:** ejemplo de uso real compuesto por un *Estímulo* (acción del actor) y una *Respuesta* (comportamiento esperado del sistema).

Identificador	RX - N		
Título			
Prioridad	☐ Alta	☐ Media	☐ Baja
Descripción			
Escenario	<i>Estímulo:</i>		
	<i>Respuesta:</i>		

TABLA 4.1. PLANTILLA DE REQUISITOS

4.1. Descripción general del sistema

El objetivo del sistema es apoyar el análisis de informes técnicos de malware mediante la clasificación binaria de frases. En este contexto, cada frase se clasifica como relevante si contiene información útil sobre el comportamiento del malware, o como no relevante si aporta contexto general, información administrativa, referencias u otros contenidos que no describen directamente la amenaza.

Este planteamiento responde a una necesidad práctica. Los informes de malware pueden ser extensos y contener mucha información heterogénea. Revisarlos manualmente requiere tiempo y puede dificultar la localización rápida de las frases que describen acciones del programa malicioso, artefactos afectados, conexiones, modificaciones del sistema o mecanismos de persistencia.

El flujo de trabajo previsto sería el siguiente. En primer lugar, el analista proporciona un informe de malware en formato texto. Después, el sistema segmenta el documento en frases y aplica el modelo de clasificación seleccionado. Finalmente, devuelve las frases clasificadas como relevantes. Estos resultados podrían utilizarse como apoyo para el análisis manual, integrarse en plataformas de inteligencia de amenazas o servir como entrada para etapas posteriores de extracción de información.

4.2. Actores y alcance

El actor principal del sistema es el **analista de ciberseguridad**, responsable de procesar informes de amenazas y extraer información relevante para alimentar sistemas de inteligencia CTI. De forma secundaria, el sistema puede ser utilizado por **investigadores de PLN** que deseen evaluar o comparar distintos modelos de clasificación sobre el corpus de SemEval-2018 Task 8 SubTask 1.

El alcance del sistema se limita a la subtarea de clasificación de relevancia (SubTask 1). No se contempla en este diseño la extracción de entidades, acciones o relaciones (SubTasks 2, 3 y 4), aunque la arquitectura está pensada para que sus salidas puedan servir como entrada a dichos módulos. [3]

4.3. Requisitos funcionales

Identificador	RF - 01
Título	Ingesta de informes de malware
Prioridad	☒ Alta ☐ Media ☐ Baja
Descripción	El sistema debe ser capaz de recibir informes técnicos de malware en formato de texto plano y segmentarlos automáticamente en frases individuales, preparándolas para su clasificación.
Escenario	<i>Estímulo:</i> el analista carga un informe APT en formato .txt en el sistema. <i>Respuesta:</i> el sistema segmenta el documento en frases, descarta cabeceras, pies de página y elementos no textuales, y devuelve una lista numerada de frases listas para clasificar.

TABLA 4.2. REQUISITO FUNCIONAL NÚMERO 01

Identificador	RF - 02
Título	Clasificación binaria de relevancia
Prioridad	☒ Alta ☐ Media ☐ Baja
Descripción	El sistema debe clasificar cada frase del informe como relevante (contiene información sobre acciones, comportamientos o características del malware) o no relevante (contexto general, texto administrativo o información no técnica), devolviendo una etiqueta binaria por frase.
Escenario	<i>Estímulo:</i> el sistema recibe la frase “ <i>The malware connects to a command and control server to receive instructions.</i> ” <i>Respuesta:</i> el sistema devuelve la etiqueta relevante (1).

TABLA 4.3. REQUISITO FUNCIONAL NÚMERO 02

Identificador	RF - 03
Título	Selección del modelo de clasificación
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	El sistema debe permitir seleccionar entre los distintos modelos evaluados en este trabajo (SVM, BiLSTM, Transformers genéricos y de dominio), de modo que el analista pueda elegir el modelo más adecuado en función del equilibrio deseado entre precisión, recall y coste computacional.
Escenario	<i>Estímulo:</i> el analista accede a la configuración del sistema y selecciona el modelo SecureBERT 2.0 para el análisis de un nuevo informe. <i>Respuesta:</i> el sistema carga el modelo seleccionado y lo aplica al informe, mostrando los resultados de clasificación junto al nombre del modelo utilizado.

TABLA 4.4. REQUISITO FUNCIONAL NÚMERO 03

Identificador	RF - 04
Título	Filtrado de frases relevantes
Prioridad	☒ Alta ☐ Media ☐ Baja
Descripción	El sistema debe permitir filtrar y visualizar únicamente las frases clasificadas como relevantes, presentándolas de forma ordenada y diferenciada del texto original, para facilitar la revisión por parte del analista.
Escenario	<i>Estímulo:</i> tras clasificar un informe de 200 frases, el analista solicita ver solo las predichas como relevantes. <i>Respuesta:</i> el sistema muestra únicamente las frases con etiqueta relevante (1), numeradas y ordenadas según su posición en el informe original, ocultando el resto.

TABLA 4.5. REQUISITO FUNCIONAL NÚMERO 04

Identificador	RF - 05
Título	Exportación de resultados
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	El sistema debe permitir exportar los resultados de clasificación en formato CSV, incluyendo el texto de cada frase y su etiqueta predicha, para facilitar su integración con otras herramientas de análisis o plataformas CTI.
Escenario	<i>Estímulo:</i> el analista solicita exportar los resultados tras clasificar un informe. <i>Respuesta:</i> el sistema genera un archivo <code>resultados.csv</code> con dos columnas (frase, etiqueta) y lo pone a disposición del analista para su descarga.

TABLA 4.6. REQUISITO FUNCIONAL NÚMERO 05

Identificador	RF - 06
Título	Evaluación del modelo sobre un conjunto etiquetado
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	El sistema debe permitir evaluar el rendimiento de un modelo dado sobre un conjunto de datos etiquetado, calculando y mostrando las métricas de precisión, recall, F1-score de la clase relevante, accuracy y Macro F1, siguiendo el protocolo de evaluación de SemEval-2018 Task 8 SubTask 1.
Escenario	<i>Estímulo:</i> el investigador proporciona el conjunto de test oficial (618 frases etiquetadas) y selecciona el modelo BiLSTM Villani para su evaluación. <i>Respuesta:</i> el sistema clasifica las 618 frases, compara las predicciones con las etiquetas reales y devuelve un informe de métricas con los valores obtenidos.

TABLA 4.7. REQUISITO FUNCIONAL NÚMERO 06

Identificador	RF - 07
Título	Registro de experimentos
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	El sistema debe almacenar un registro de los experimentos realizados, incluyendo el modelo utilizado, los hiperparámetros configurados, la fecha de ejecución y las métricas obtenidas, permitiendo la reproducibilidad y la comparación entre experimentos.
Escenario	<i>Estímulo:</i> el investigador ejecuta una evaluación con cualquiera de los modelos disponibles en el sistema (SVM, BiLSTM o Transformer). <i>Respuesta:</i> el sistema guarda automáticamente una entrada en el registro de experimentos con el nombre del modelo, la configuración completa y los resultados obtenidos.

TABLA 4.8. REQUISITO FUNCIONAL NÚMERO 07

4.4. Requisitos no funcionales

Identificador	RNF - 01
Título	Rendimiento en inferencia
Prioridad	☒ Alta ☐ Media ☐ Baja
Descripción	El sistema debe ser capaz de clasificar un informe de tamaño medio (200–300 frases) en un tiempo razonable. Los modelos basados en SVM o BiLSTM deben completar la clasificación en menos de 30 segundos. Los modelos Transformer pueden requerir más tiempo, pero no deben superar los 5 minutos en hardware con GPU disponible.
Escenario	<i>Estímulo:</i> el analista lanza la clasificación de un informe de 250 frases con el modelo SVM optimizado. <i>Respuesta:</i> el sistema completa la clasificación y muestra los resultados en menos de 10 segundos.

TABLA 4.9. REQUISITO NO FUNCIONAL NÚMERO 01

Identificador	RNF - 02
Título	Reproducibilidad de los experimentos
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	El sistema debe facilitar la reproducibilidad de los experimentos mediante la publicación del código fuente, los datasets preprocesados y las configuraciones de hiperparámetros en el repositorio público del proyecto. Dado que los experimentos no fijan semilla aleatoria de forma sistemática, los resultados numéricos pueden presentar variaciones menores entre ejecuciones, aunque el comportamiento general y el ranking entre modelos deben mantenerse estables.
Escenario	<i>Estímulo:</i> un investigador externo descarga el repositorio del proyecto y ejecuta el notebook de evaluación del SVM con GridSearch. <i>Respuesta:</i> el sistema reproduce el experimento con los mismos datos y configuración de hiperparámetros, obteniendo un F1 de la clase relevante en el rango reportado en la Tabla 7.1, con posibles variaciones menores debidas a la aleatoriedad no controlada.

TABLA 4.10. REQUISITO NO FUNCIONAL NÚMERO 02

Identificador	RNF - 03
Título	Modularidad y extensibilidad
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	El sistema debe estar organizado de forma modular, separando claramente las etapas de preprocesamiento, entrenamiento, evaluación y exportación de resultados. Esta estructura debe facilitar la incorporación de nuevos modelos o técnicas de aumento de datos sin necesidad de modificar los módulos existentes.
Escenario	<i>Estímulo:</i> un investigador desea añadir un nuevo modelo Transformer al sistema. <i>Respuesta:</i> el investigador puede incorporarlo implementando únicamente el módulo de modelo correspondiente, sin alterar el código de preprocesamiento ni el de evaluación.

TABLA 4.11. REQUISITO NO FUNCIONAL NÚMERO 03

Identificador	RNF - 04
Título	Compatibilidad con el entorno de experimentación
Prioridad	☒ Alta ☐ Media ☐ Baja
Descripción	El sistema debe ser compatible con el entorno de ejecución utilizado durante el desarrollo del proyecto: Python 3 sobre Google Colab con GPU T4 o P100 (16 GB de VRAM). Las dependencias deben especificarse con versiones concretas para evitar incompatibilidades, y el sistema debe poder ejecutarse sin configuración local adicional mediante notebooks de Jupyter.
Escenario	<i>Estímulo:</i> el investigador abre el notebook de entrenamiento de BiLSTM en Google Colab. <i>Respuesta:</i> el sistema instala automáticamente las dependencias especificadas, carga el dataset desde HuggingFace y ejecuta el entrenamiento sin errores de compatibilidad.

TABLA 4.12. REQUISITO NO FUNCIONAL NÚMERO 04

Identificador	RNF - 05
Título	Documentación del código
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	Todo el código fuente del sistema debe estar documentado con comentarios explicativos en las funciones y bloques principales, de modo que un desarrollador con conocimientos de Python y PLN pueda comprender su funcionamiento sin necesidad de consultar documentación externa.
Escenario	<i>Estímulo:</i> un nuevo colaborador del proyecto accede al repositorio por primera vez y abre el notebook de preprocesamiento. <i>Respuesta:</i> el colaborador comprende el flujo de procesamiento sin dificultad gracias a los comentarios y celdas de texto explicativo presentes en el notebook.

TABLA 4.13. REQUISITO NO FUNCIONAL NÚMERO 05

Identificador	RNF - 06
Título	Trazabilidad de los datos
Prioridad	☐ Alta ☒ Media ☐ Baja
Descripción	El sistema debe garantizar que los datos utilizados en cada experimento sean rastreables hasta su fuente original. El dataset preprocesado debe estar publicado con su procedencia documentada (repositorio SecureNLP de SemEval-2018 Task 8), y cualquier modificación aplicada al dataset original debe estar explícitamente descrita en la documentación del proyecto.
Escenario	<i>Estímulo:</i> un revisor desea verificar la procedencia de los datos de entrenamiento utilizados en el experimento de back-translation. <i>Respuesta:</i> el sistema documenta que las frases base provienen del split de entrenamiento de MalwareTextDB, publicado en el repositorio SecureNLP, y que el dataset preprocesado está disponible en https://huggingface.co/datasets/hugobecerra/DATASET3.0.

TABLA 4.14. REQUISITO NO FUNCIONAL NÚMERO 06

5. DATASET Y PREPROCESAMIENTO

Este capítulo describe en detalle el dataset utilizado en la experimentación, así como el proceso de obtención y preparación de los datos, y un análisis de sus características cuyo fin es el de ayudar a entender los resultados posteriores. Todo el código fuente, *notebooks* y *scripts* desarrollados para el preprocesamiento y experimentación descritos en este documento han sido publicados bajo licencia de código abierto y pueden consultarse en el repositorio oficial del proyecto³.

Los datos utilizados en este trabajo provienen de la competición SemEval-2018 Task 8 [3], concretamente de la SubTask 1 (clasificación de relevancia). El dataset en el que se basa es MalwareTextDB [4], descrito en el Capítulo 3.

El esquema de anotación, basado en el vocabulario MAEC [56], se describe en el Capítulo 3.

Para la SubTask 1, una frase es considerada **relevante** si contiene al menos una anotación de cualquiera de estos tipos (es decir, si describe alguna característica o comportamiento del malware). El resto de frases, como descripciones de la víctima o texto administrativo se etiquetan como **no relevantes**.

Para este trabajo, se obtuvo el dataset del repositorio público de SecureNLP⁴ y se desarrolló un *script* de preprocesamiento propio en Python (disponible en el repositorio del proyecto) para convertirlo al formato necesario para clasificación binaria. El proceso fue el siguiente:

1. **Extracción:** Se descargaron los archivos tokenizados de las particiones (*splits*) de entrenamiento, desarrollo y test.
2. **Reconstrucción de frases:** Los archivos originales contienen un token por línea con su etiqueta BIO. Se agruparon los tokens por frase usando los separadores del formato original.
3. **Asignación de etiquetas:** Para cada frase, se determinó su etiqueta de relevancia con la siguiente lógica:
 - Si **todos** los tokens tienen etiqueta “O” (Outside), la frase es **no relevante** (0).
 - Si **al menos un** token tiene una etiqueta diferente de “O” (es decir, está marcado como Acción, Entidad o Modificador), la frase es **relevante** (1).
4. **Generación de CSV:** Se exportaron los datos a archivos CSV con tres columnas: identificador único, texto de la frase, y etiqueta binaria.

³<https://github.com/100495717/TFG-HUGO-BECERRA>

⁴<https://github.com/anuraagkansara/SecureNLP>

El dataset procesado resultante se publicó en HuggingFace⁵ para facilitar su uso en los distintos notebooks del trabajo.

5.1. Distribución de clases

Los splits de entrenamiento, desarrollo y test fueron proporcionados por los organizadores de la competición y se obtuvieron del repositorio público de SecureNLP⁶. La Tabla 5.1 muestra la distribución de clases en cada partición tras aplicar el proceso de preprocesamiento descrito anteriormente:

Split	Total	Relevantes	No relevantes	Ratio
Train	9.435	2.204 (23,4 %)	7.231 (76,6 %)	3,28:1
Dev	1.213	79 (6,5 %)	1.134 (93,5 %)	14,35:1
Test	618	90 (14,6 %)	528 (85,4 %)	5,87:1
Total	11.266	2.373 (21,1 %)	8.893 (78,9 %)	3,75:1

TABLA 5.1. DISTRIBUCIÓN DEL DATASET POR SPLITS

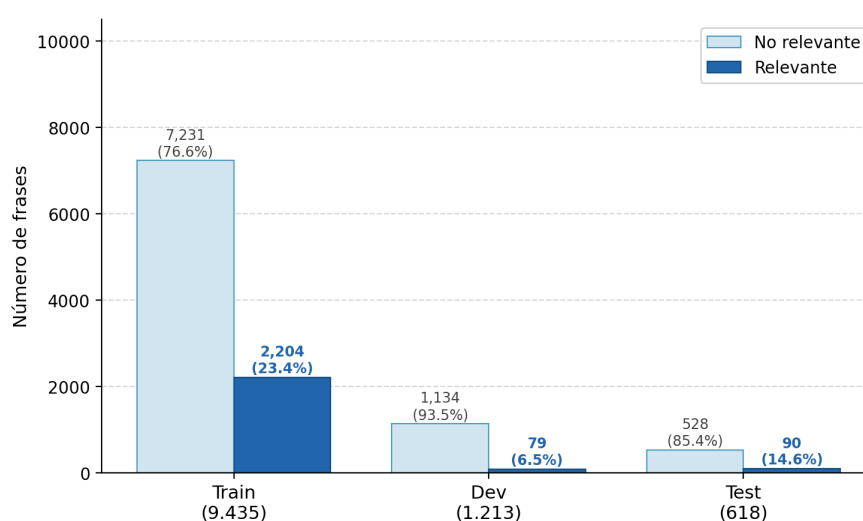


Fig. 5.1. Distribución de clases en los conjuntos de entrenamiento (train+dev fusionados, 10.648 frases) y test (618 frases) del corpus SemEval-2018 Task 8 SubTask 1. El desequilibrio es notable en ambos subconjuntos, con la clase relevante representando el 23,4 % en entrenamiento y el 14,6 % en test. Elaboración propia.

Cabe mencionar aspectos claves sobre estos datos:

Desbalanceo significativo. La clase mayoritaria (no relevante) supera con creces a la minoritaria (relevante) en los tres conjuntos. Este desbalanceo se acentúa en el conjun-

⁵<https://huggingface.co/datasets/hugobecerra/DATASET3.0>

⁶<https://github.com/anuraagkansara/SecureNLP>

to Dev (desarrollo), donde hay más de 14 frases no relevantes por cada frase relevante. Esto muestra la realidad de los informes de malware: la mayor parte del texto es contexto, explicaciones o información administrativa, y solo una pequeña fracción describe comportamientos técnicos específicos del malware.

Inconsistencia entre particiones. Los ratios de desbalanceo varían notablemente (3,28:1 en entrenamiento, 13,35:1 en desarrollo, 5,87:1 en test). Esto sugiere que las particiones no se crearon mediante un muestreo estratificado a nivel de frase, sino probablemente separando informes completos. Como consecuencia, un modelo entrenado en la partición de entrenamiento puede encontrarse con una distribución muy diferente en la evaluación.

Tamaño moderado. El split de entrenamiento contiene 9.435 frases. Para entrenar modelos de *deep learning*, esta cantidad es reducida, sobre todo si se emplean arquitecturas con muchos parámetros. En este escenario aumenta el riesgo de sobreajuste y cobra más peso el uso de regularización, transferencia de aprendizaje o modelos preentrenados. Esta relación entre tamaño del corpus, capacidad del modelo y generalización está documentada en la literatura de clasificación de textos y aprendizaje profundo [22], [27].

Para entender mejor la naturaleza de los datos, se analizaron cualitativamente ejemplos representativos de cada clase.

Las frases relevantes describen comportamientos concretos del malware:

“The malware connects to a command and control server to receive instructions.”

“Upon execution, the Trojan drops a DLL file in the Windows system directory.”

“The ransomware encrypts files with the following extensions: .doc, .pdf, .xls.”

Estas frases contienen verbos de acción (*connects*, *drops*, *encrypts*) y entidades específicas (*command and control server*, *DLL file*, *Windows system directory*). Son exactamente el tipo de información que un analista de seguridad desearía extraer automáticamente para generar inteligencia accionable.

Las frases no relevantes incluyen contexto general, información administrativa o descripciones que no aportan datos técnicos sobre la amenaza:

“This report was prepared by the security research team.”

“The campaign targeted organizations in the financial sector.”

“Figure 3 shows the infection chain.”

Existen frases que se encuentran en la frontera entre ambas clases y cuya clasificación puede resultar discutible, incluso para anotadores humanos:

“The attackers used spear-phishing emails to deliver the payload.”

Esta frase describe una técnica de ataque o vector de infección inicial (*spear-phishing*), pero no detalla un comportamiento específico del malware una vez en el sistema. Según el esquema de anotación MAEC, podría considerarse relevante o no dependiendo de la amplitud de la interpretación. Estos casos límite contribuyen a la dificultad intrínseca de la tarea y establecen un límite superior realista para el rendimiento de cualquier sistema automatizado.

Además del desbalanceo entre clases, conviene revisar la longitud de las frases en cada partición. Esta longitud, medida en tokens, afecta directamente a dos decisiones del trabajo: el límite de truncamiento utilizado en los modelos Transformer y la capacidad de los modelos recurrentes para procesar frases más largas.

También resulta útil comparar la longitud de las frases relevantes y no relevantes. Si las frases relevantes fueran claramente más largas, el truncamiento podría afectar más a la clase positiva. En este trabajo, el límite se fijó en 128 tokens para los modelos Transformer.

Esta decisión se tomó por dos motivos. En primer lugar, las frases de los informes sobre amenazas persistentes avanzadas (APT, *Advanced Persistent Threat*) suelen ser sentencias técnicas de longitud moderada. En trabajos de clasificación textual en ciberseguridad con corpus similares también se ha utilizado un límite de 128 tokens, suficiente para cubrir la mayoría de las instancias analizadas [42], [63].

En segundo lugar, Sun et al. [35] muestran que, en tareas de clasificación de texto con BERT, utilizar los primeros 128 tokens puede ser una estrategia competitiva frente a alternativas más costosas. Además, sus resultados indican que otros hiperparámetros, como la tasa de aprendizaje o el número de épocas, pueden tener más impacto en el rendimiento final que aumentar la longitud máxima de entrada.

El corpus de SemEval-2018 Task 8 SubTask 1 presenta características que lo convierten en un marco de evaluación desafiante pero realista para la clasificación de textos en el dominio de la ciberseguridad:

- Un léxico técnico altamente especializado, sugiriendo ventajas competitivas para los modelos preentrenados específicamente en el dominio CTI.
- La existencia de casos ambiguos, que penalizan a los modelos puramente estadísticos.

Estas particularidades estructuran y justifican la exploración metodológica desarrollada en el siguiente capítulo.

6. ENFOQUES

Este capítulo describe los modelos implementados para abordar la clasificación de relevancia de frases extraídas de informes técnicos de malware (SemEval-2018 Task 8 SubTask 1). Se comparan distintas familias de modelos con el fin de evaluar si el incremento de complejidad arquitectónica, el preentrenamiento especializado o el aumento de datos producen mejoras en esta tarea.

La experimentación se organizó en cinco familias. En primer lugar se implementó un clasificador SVM con representación TF-IDF como sistema baseline. A continuación se utilizaron modelos recurrentes BiLSTM con atención, después Transformers genéricos y Transformers preentrenados con textos del dominio en ciberseguridad, y finalmente se exploró la clasificación sin fine-tuning mediante *prompting*. Adicionalmente, se aplicó aumento de datos mediante *back-translation* sobre BERT y SecureBERT: se eligieron estos dos modelos porque representan, respectivamente, el Transformer genérico de referencia y el modelo de dominio basado en RoBERTa, lo que permite evaluar el efecto del aumento sobre arquitecturas con distintos niveles de especialización en ciberseguridad.

Todo el código está disponible en el repositorio público del proyecto⁷.

6.1. Diseño experimental

Los experimentos parten de las particiones oficiales de SecureNLP SubTask 1, descritas en el Capítulo 5. La evaluación final de todos los modelos se realizó exclusivamente sobre el conjunto de test (618 frases), que no se utilizó en ningún momento durante el entrenamiento ni durante la selección de configuraciones.

Para todos los modelos supervisados se fusionaron los conjuntos de entrenamiento y desarrollo en un único corpus de 10.648 frases. Esta decisión responde a dos razones. Primera, el tamaño moderado del corpus hace aconsejable aprovechar todos los datos etiquetados disponibles para el ajuste de los modelos, especialmente en la clase relevante, que representa apenas el 23,4 % del split de entrenamiento oficial. Segunda, el conjunto de desarrollo presenta un desbalanceo severo (14,35:1), muy distinto al de test (5,87:1), lo que lo hace poco representativo como partición de validación independiente: las métricas obtenidas sobre él no reflejan con fidelidad el comportamiento esperado en el conjunto de evaluación final. En el caso de la BiLSTM, el criterio de parada temprana (*early stopping*) se aplicó monitorizando la pérdida sobre un subconjunto del propio corpus de entrenamiento reservado internamente, sin recurrir al conjunto de desarrollo.

Como punto de comparación, BERT se evaluó adicionalmente bajo una segunda configuración —denominada Configuración B— entrenada únicamente sobre el split de en-

⁷<https://github.com/100495717/TFG-HUGO-BECERRA>

trenamiento (9.435 frases), con hiperparámetros idénticos ($lr = 5 \times 10^{-5}$, 3 épocas, lote 8, sin selección de *checkpoint*) y evaluada igualmente sobre el conjunto de test. El objetivo fue cuantificar empíricamente el efecto de incluir el conjunto de desarrollo en el entrenamiento. El protocolo estándar con fusión train+dev se denomina Configuración A cuando se hace referencia a BERT en el análisis comparativo.

La métrica principal es el F1-score de la clase relevante —es decir, las frases que describen acciones o comportamientos del malware—, siguiendo el criterio de evaluación oficial de la competición [3].

6.1.1. Tratamiento del desbalanceo

El desbalanceo se abordó de forma distinta según el modelo. En el SVM con búsqueda de hiperparámetros se incluyó `class_weight` como parámetro de exploración: este parámetro asigna mayor peso a los errores cometidos sobre la clase minoritaria durante el entrenamiento, de modo que el clasificador penaliza más los fallos sobre las frases relevantes. La configuración óptima encontrada dejó este parámetro en `None`, lo que indica que dicho ajuste no aportó mejora en este caso. En los modelos neuronales no se generaron ejemplos artificiales de la clase minoritaria ni se descartaron ejemplos de la mayoritaria para equilibrar las clases durante el entrenamiento.

6.1.2. Entorno de experimentación

Toda la experimentación se ejecutó en Google Colab sobre GPUs T4 o P100, con un límite de 16 GB de VRAM. Esta restricción condicionó varias decisiones: el tamaño de lote reducido (8 en la mayoría de modelos) y la imposibilidad de cargar LLMs de gran tamaño. El truncamiento a 128 tokens aplicado en los Transformers se justifica además por las características del corpus: como se analiza en el Capítulo 5, el 95 % de las frases del dataset no supera este límite, por lo que el truncamiento no afecta de forma significativa a la información disponible para el modelo. El ecosistema de software se basó en Python, con *scikit-learn* [12] para los modelos estadísticos, y *PyTorch* [13] con la librería *Transformers* de HuggingFace [14] para los modelos de aprendizaje profundo.

6.2. Baseline: SVM con TF-IDF

Como punto de partida se implementó un clasificador SVM (Support Vector Machine) lineal con representación TF-IDF, acorde al enfoque del baseline oficial de la competición [3].

6.2.1. Representación del texto

Las frases se representaron utilizando el `TfidfVectorizer` de *scikit-learn*. La extracción de características contempló unigramas y bigramas para retener palabras individuales y expresiones multipalabra frecuentes, como “*command and*” o “*registry key*”. Para reducir la influencia de términos muy repetidos dentro de un mismo documento, se aplicó un escalado sublineal de frecuencias (`sublinear_tf=True`), que transforma la frecuencia bruta tf en $1 + \log(tf)$. Se descartaron términos con frecuencia mínima de documento inferior a 2 y términos presentes en más del 90 % del corpus. Además, se eliminaron las *stopwords* estándar del inglés.

6.2.2. Clasificador y optimización

El algoritmo seleccionado fue `LinearSVC`. Frente a la opción de utilizar kernels polinómicos o radiales (RBF), el kernel lineal proporciona buenos resultados en clasificación de textos: en espacios TF-IDF con decenas de miles de dimensiones (una por n-grama), los datos tienden naturalmente a la separabilidad lineal, reduciendo drásticamente la carga computacional y el riesgo de sobreajuste [23].

Para maximizar la capacidad predictiva del clasificador, se ejecutó una búsqueda en malla (`GridSearchCV`) acoplada a una validación cruzada estratificada de 5 particiones. Los parámetros explorados abarcaron:

- Rango de n-gramas: {(1, 1), (1, 2)}
- Frecuencia máxima de documento (`max_df`): {0,9, 1,0}
- Frecuencia mínima de documento (`min_df`): {1, 2}
- Penalización L2 (C): {0,25, 0,5, 1,0, 2,0}
- Ponderación de clases: {None, balanced}

De las 64 combinaciones evaluadas, la mejor configuración obtuvo un F1 medio en validación cruzada de 0,6155. Los parámetros seleccionados fueron: $C = 1,0$, rango de n-gramas (1,2), `max_df=0.9`, `min_df=1` y `class_weight=None`. Aunque se probó `class_weight='balanced'` para compensar el desbalanceo de clases, esta opción no mejoró el rendimiento. En este caso, aumentar el peso de la clase minoritaria tendía a favorecer el *recall*, pero reducía la precisión del clasificador.

6.3. BiLSTM con atención y GloVe

El segundo enfoque consistió en replicar la arquitectura del equipo Villani [5], ganador de la SubTask 1 en SemEval-2018. El objetivo era comprobar hasta qué punto podía

reproducirse su resultado y utilizarlo como referencia frente al resto de modelos evaluados.

Para ello se trabajó con dos variantes BiLSTM. La primera fue una versión inicial con dos capas bidireccionales consecutivas. La representación de entrada se construyó con vectores preentrenados GloVe de 300 dimensiones (`glove.6B.300d`), congelados durante el entrenamiento, y con el vocabulario limitado a las 20.000 palabras más frecuentes. La primera capa procesó las secuencias con 128 unidades por dirección y devolvió la secuencia completa a una segunda capa de 64 unidades por dirección. Después se aplicó una capa de atención basada en tangente hiperbólica y normalización temporal con *softmax*. Esta red se optimizó con Adam, una tasa de aprendizaje de 8×10^{-5} y un máximo de 15 épocas.

La segunda variante replica con mayor fidelidad la propuesta de Villani [5]. La principal diferencia respecto a la primera variante es la representación de entrada: en lugar de usar únicamente embeddings GloVe de palabras completas, se concatenaron vectores GloVe de 300 dimensiones con embeddings de trigramas de caracteres de 200 dimensiones. Esta combinación permite representar palabras completas y, al mismo tiempo, capturar patrones internos de los términos, algo útil en textos de ciberseguridad donde aparecen rutas, ejecutables, librerías, dominios y nombres poco frecuentes.

A nivel arquitectónico, esta segunda variante utilizó una única capa BiLSTM con 256 unidades por dirección y dropout de 0.3.

Sobre los estados ocultos de la BiLSTM se aplicó un mecanismo de atención, que asigna un peso a cada posición de la secuencia para calcular una representación ponderada de la frase completa. En lugar de tomar simplemente el último estado oculto o un promedio de todos ellos, la atención permite que el modelo focalice su representación en los tokens más informativos para la clasificación.

Se utilizó la atención de Luong [5], basada en producto escalar entre el estado oculto final y cada estado intermedio. Este mecanismo es más sencillo computacionalmente que la atención de Bahdanau [64], que calcula la similitud mediante una red neuronal adicional. Ambos producen resultados similares en tareas de clasificación de texto, pero el producto escalar es más eficiente con secuencias cortas como las del corpus.

La optimización se realizó con RMSprop (*Root Mean Square Propagation*), un optimizador con tasa de aprendizaje adaptativa que suele funcionar bien en redes recurrentes. Se utilizó una tasa de aprendizaje inicial de 10^{-4} , *early stopping* con paciencia de 2 épocas mediante la estrategia ReduceLROnPlateau, que reduce la tasa de aprendizaje cuando la pérdida de validación se estanca durante varias épocas consecutivas. Esto permite continuar el entrenamiento con ajustes más pequeños en las fases finales.

6.4. Transformers genéricos: BERT, RoBERTa y DeBERTa

Esta familia recoge modelos Transformer *encoder-only* preentrenados sobre grandes corpus de texto general, sin especialización en ciberseguridad. Se seleccionaron tres arquitecturas ampliamente empleadas en tareas de clasificación de textos en el dominio de la seguridad y otros dominios técnicos: BERT [6], RoBERTa [8] y DeBERTa v3 [7], todos ellos utilizados como referencia en trabajos recientes de PLN aplicado a ciberseguridad [9], [43].

- **BERT base** (`bert-base-uncased`): modelo encoder propuesto por Devlin et al. [6], con 110 millones de parámetros. Se utilizó como referencia principal dentro de los Transformers genéricos.
- **RoBERTa base** (`roberta-base`): variante de BERT propuesta por Liu et al. [8], entrenada con más datos y sin la tarea de predicción de la siguiente frase. En este trabajo se incluye para comprobar si estas mejoras de preentrenamiento se trasladan a la clasificación de frases de malware.
- **DeBERTa v3 base** (`microsoft/deberta-v3-base`): modelo propuesto por He et al. [7], que separa la representación del contenido y la posición de los tokens. Se evaluó como una arquitectura Transformer más reciente dentro de los modelos generales.

El fine-tuning se planteó como una tarea de clasificación binaria. Para ello, se añadió una capa lineal de clasificación sobre la representación final del token [CLS] o su equivalente según el modelo. El texto se tokenizó con el tokenizador propio de cada arquitectura, con truncamiento a 128 tokens. Este límite está justificado por la distribución de longitudes del corpus: como se detalla en el Capítulo 5, el 95 % de las frases no supera este umbral, por lo que el truncamiento apenas descarta información relevante.

Para BERT, RoBERTa, DeBERTa v3 y los Transformers de dominio se utilizó un protocolo común: tasa de aprendizaje 5×10^{-5} , tamaño de lote 8, tres épocas y `weight_decay` de 0,01, sin selección de *checkpoint*. Esta configuración sigue las recomendaciones habituales para el fine-tuning de modelos BERT-like en tareas de clasificación con datasets de tamaño moderado [6], [8]. Todos estos modelos se entrenaron sobre la fusión train+dev (10.648 frases), con la excepción de BERT Configuración B, que utilizó únicamente el split oficial (9.435 frases) para cuantificar el efecto del volumen de datos; sus implicaciones se discuten en el Capítulo 7.

6.5. Transformers de ciberseguridad

Este trabajo tiene como una de las preguntas más importantes si los modelos Transformer preentrenados en textos de ciberseguridad ofrecen ventajas sobre los Transformers

genéricos en esta tarea en específico. Es por ello que se han evaluado tres modelos de dominio disponibles de forma pública.

Se evaluaron tres modelos Transformer especializados en ciberseguridad disponibles públicamente. El protocolo de fine-tuning es el mismo descrito en la sección anterior (tasa de aprendizaje 5×10^{-5} , lote 8, tres épocas); la única diferencia respecto a los Transformers genéricos es el modelo de partida.

CTI-BERT (ibm-research/CTI-BERT). Desarrollado por IBM Research [9]. Parte de los pesos preentrenados de BERT-base y continúa el preentrenamiento mediante MLM con un corpus de textos de inteligencia de amenazas (CTI), boletines de seguridad y documentación de vulnerabilidades. Este proceso de preentrenamiento continuo adapta las representaciones de BERT al vocabulario y las construcciones propias de los informes de ciberseguridad, sin modificar su arquitectura.

SecureBERT 2.0 (cisco-ai/SecureBERT2.0-base). Modelo desarrollado por Cisco AI [10] y basado en ModernBERT, una revisión reciente de la arquitectura BERT que incorpora atención rotacional de posición (*RoPE*), ventanas de atención local alternadas con atención global y mayor eficiencia para secuencias largas, en lugar de los embeddings de posición absoluta y la atención cuadrática completa del BERT original. A diferencia del SecureBERT original [43], que partía de RoBERTa y se preentrenaba con texto de ciberseguridad, SecureBERT 2.0 actualiza tanto la arquitectura base como el corpus, que combina textos de ciberseguridad y código fuente.

SecBERT (nlpaueb/sec-bert-base). Modelo de arquitectura BERT-base [11], preentrenado desde cero —sin partir de los pesos de BERT ni RoBERTa— con un vocabulario y un corpus específicos del dominio de ciberseguridad. Se encuentra de forma pública en HuggingFace.

Estos modelos han sido seleccionados porque, al haber sido entrenados con grandes cantidades de texto de ciberseguridad, habrían aprendido mejor la terminología, las construcciones sintácticas y las relaciones semánticas del dominio. Por ejemplo, es menos probable que un modelo genérico pueda asociar correctamente “drops a DLL” con un comportamiento de malware, mientras que un modelo de dominio habría visto esta expresión frecuentemente y la codificaría de forma más apropiada con más certeza.

6.6. Modelos de lenguaje grandes con *prompting*

Como alternativa al fine-tuning supervisado, se evaluó el uso de modelos de lenguaje grandes mediante *prompting*. En este enfoque no se reentrenan los parámetros del modelo: la tarea se formula directamente en lenguaje natural, proporcionando al modelo una instrucción y, cuando procede, algunos ejemplos dentro del propio *prompt*. Esta forma

de uso se ha extendido en tareas de Procesamiento del Lenguaje Natural porque permite adaptar un modelo a una tarea concreta sin modificar sus pesos [44], [51].

6.6.1. Clasificación por inferencia textual (BART-MNLI)

El primer enfoque utilizó facebook/bart-large-mnli [39], una versión de BART ajustada sobre MNLI (*Multi-Genre Natural Language Inference*) [65]. La inferencia de lenguaje natural consiste en determinar la relación entre dos textos: una premisa y una hipótesis. En este caso, la frase del informe de malware se utilizó como premisa. La clasificación se reformuló mediante dos hipótesis candidatas. Por ejemplo, para la frase:

“The malware connects to a remote server to receive commands.”

se construyeron las hipótesis: *“Esta frase describe un comportamiento de malware”* (clase relevante) y *“Esta frase no describe un comportamiento de malware”* (clase no relevante). El modelo predice la relación entre la premisa y cada hipótesis, y la clase con mayor puntuación de *entailment* es la predicción final.

Este enfoque no requiere ejemplos etiquetados de la tarea, pero su rendimiento depende de cómo se formulen las hipótesis: una redacción imprecisa puede sesgar las predicciones hacia una clase, especialmente cuando el corpus está tan desbalanceado como este.

6.6.2. Clasificación mediante instrucciones (FLAN-T5)

El segundo enfoque utilizó google/flan-t5-base [50], un modelo ajustado para seguir instrucciones en lenguaje natural. En este caso, el *prompt* incluía la instrucción de clasificar una frase como relevante o no relevante para la caracterización de malware, seguida de dos ejemplos ya etiquetados extraídos del conjunto de entrenamiento: uno de la clase relevante y otro de la clase no relevante. Después de esos ejemplos, se añadía la frase del test que el modelo debía clasificar. El *prompt* completo utilizado fue el siguiente:

Classify the following sentence as Relevant or Not relevant
for malware characterization.

Sentence: "The malware encrypts files on the infected system."
Classification: Relevant

Sentence: "The report was published by the security team."
Classification: Not relevant

Sentence: "[frase a clasificar]" ->

Este experimento debe tomarse como una exploración preliminar. FLAN-T5-base tiene 250 millones de parámetros, muy por debajo de los modelos que han mostrado capacidades competitivas de clasificación sin back-translation en la literatura [44], [46]. Se eligió porque era el modelo más grande ejecutable en el entorno disponible (16 GB de VRAM). Modelos actuales como LLaMA-3 o GPT-5 requerirían infraestructura no disponible en este trabajo.

6.7. Aumento de datos: Traducción Circular (Back-translation)

Enfrentar un ratio de desbalanceo muy grande requiere en ocasiones alterar el dataset original. Se descartó el uso de SMOTE (*Synthetic Minority Over-sampling Technique*) porque su aplicación al texto resulta complicada. Esta técnica crea nuevos ejemplos interpolando vectores de características, pero esos puntos intermedios no tienen por qué corresponderse con frases reales ni mantener un significado claro. Por este motivo, podía introducir ruido en el entrenamiento en lugar de aportar ejemplos útiles para la clase relevante, optando en su lugar por técnicas de aumento basadas en traducción automática [55].

La *back-translation* consiste en traducir una frase a un idioma intermedio y volver después al idioma original para obtener una reformulación automática del texto [52], [55]. En este trabajo se utilizó francés como idioma intermedio. Primero se tradujeron frases relevantes del inglés al francés con `Helsinki-NLP/opus-mt-en-fr`, un modelo de traducción automática neuronal basado en la arquitectura Transformer y entrenado sobre el corpus OPUS [66], disponible públicamente en HuggingFace, y después se tradujeron de nuevo al inglés con el modelo inverso. El objetivo era generar variantes de frases relevantes sin cambiar, en la medida de lo posible, su significado original.

Se evaluó su impacto en dos escenarios:

1. **Aumento controlado aplicado a BERT:** se seleccionaron 300 frases relevantes de la fusión train+dev, se generó una variante por back-translation de cada una, y se añadieron al corpus de entrenamiento de BERT (Configuración A). El objetivo era comprobar si una ampliación acotada de la clase minoritaria mejoraba el rendimiento sin alterar la distribución global del corpus de forma drástica.
2. **Aumento completo con SecureBERT:** se aplicó la misma técnica sobre las 2.283 frases relevantes disponibles en el corpus de entrenamiento utilizado. Este experimento se realizó con `ehsanaghaei/SecureBERT`, es decir, con el SecureBERT original basado en RoBERTa [43], no con SecureBERT 2.0.

El hecho de haber aplicado el aumento sobre modelos base distintos (BERT y SecureBERT original) impide comparar directamente el efecto del aumento de datos de forma aislada. Esta limitación se discute en el Capítulo 7.

6.8. Resumen de configuraciones

En la Tabla 6.1 se pueden ver las configuraciones de todos los modelos.

Familia	Modelo	Datos de entrenamiento	lr / épocas
Clásico	SVM	Train+Dev (10.648)	—
	SVM + GridSearch	Train+Dev (10.648)	—
Recurrente	BiLSTM (2 capas)	Train+Dev (10.648)	8×10^{-5} / 15
	BiLSTM (Villani)	Train+Dev (10.648)	10^{-4} / 5
Transformer genérico	BERT (Config. A)	Train+Dev (10.648)	5×10^{-5} / 3
	BERT (Config. B)	Train (9.435)	5×10^{-5} / 3
	RoBERTa	Train+Dev (10.648)	5×10^{-5} / 3
	DeBERTa v3	Train+Dev (10.648)	5×10^{-5} / 3
	BERT + BT	Train+Dev + 300 aug.	5×10^{-5} / 3
	CTI-BERT	Train+Dev (10.648)	5×10^{-5} / 3
Transformer dominio	SecureBERT 2.0	Train+Dev (10.648)	5×10^{-5} / 3
	SecBERT	Train+Dev (10.648)	5×10^{-5} / 3
	SecureBERT + BT	Train+Dev + 2.283 aug.	5×10^{-5} / 3
Sin entrenamiento	BART-MNLI (zero-shot)	—	—
	FLAN-T5 (2-shot)	—	—

Todos los Transformers se entrenaron con lr 5×10^{-5} , 3 épocas y lote 8, sin selección de *checkpoint*; el modelo guardado corresponde al estado final. La Configuración A utiliza la fusión train+dev; la Configuración B solo el split oficial de entrenamiento.

TABLA 6.1. RESUMEN DE MODELOS EVALUADOS

La diferencia de tasas de aprendizaje entre familias no es arbitraria. La BiLSTM se entrena desde cero y admite tasas más altas (10^{-4}) y más épocas. Los Transformers pre-entrenados requieren actualizaciones pequeñas (10^{-5}) y pocas épocas: sobrepasar este margen puede favorecer el denominado olvido catastrófico (*catastrophic forgetting*) [67], degradando las representaciones aprendidas durante el preentrenamiento.

7. RESULTADOS Y DISCUSIÓN

En este capítulo se presentan y analizan los resultados obtenidos por los modelos descritos en el Capítulo 6. Los resultados se organizan por familias, se sitúan frente a los sistemas de la competición original y se discuten los patrones que emergen del conjunto de experimentos.

7.1. Métricas utilizadas

La evaluación se ha realizado utilizando el mismo protocolo que se utiliza en la competición. Todos los modelos se evalúan sobre el conjunto de test (618 frases, de las cuales 90 son relevantes) y se reportan cinco métricas:

La **precisión** mide qué proporción de las frases predichas como relevantes lo son realmente. Se calcula como $P = TP/(TP + FP)$, donde TP son los True Positives (verdaderos positivos) y FP los falsos positivos. Una precisión alta indica que cuando el modelo asegura que una frase es relevante, acierta la mayoría de las veces.

El **recall** (también llamado sensibilidad o exhaustividad) mide qué proporción de las frases relevantes reales ha conseguido identificar el modelo. Se calcula como $R = TP/(TP + FN)$ donde FN son los falsos negativos. Un recall alto indica que el modelo no deja escapar muchas frases relevantes.

El **F1-score** es la media armónica de precisión y recall: $F1 = 2 \cdot P \cdot R / (P + R)$. Penaliza los desequilibrios entre ambas y da una visión conjunta del rendimiento del clasificador.

La **accuracy** (exactitud) mide la fracción de predicciones correctas sobre el total. Su utilidad en este escenario es limitada: como ya se discutió en el Capítulo 5, un clasificador trivial que siempre predijera alcanzaría un 85,4 % de accuracy sobre el test sin haber aprendido nada útil. Por eso se reporta junto al **Macro F1**, que promedia el F1 de ambas clases por igual y da una idea más equilibrada del comportamiento global.

La métrica principal, siguiendo el criterio fijado por los organizadores en [3], es el **F1-score de la clase relevante**. Por este motivo, el análisis posterior se centra en esta métrica. El resto de métricas se reportan como información complementaria.

7.2. Resultados

Los quince modelos evaluados (catorce de ellos con evaluación cuantitativa completa) se agrupan en cinco bloques que se corresponden con las familias presentadas en el Capítulo 6: enfoques estadísticos clásicos, redes recurrentes, Transformers genéricos, Transformers de dominio y modelos basados en prompting. La Tabla 7.1 consolida to-

dos los resultados sobre el conjunto de test, con precisión y recall específicos de la clase minoritaria (relevante).

Modelo	P	R	F1 (rel.)	Acc.	Macro F1
<i>Modelos clásicos</i>					
SVM (baseline)	0,3869	0,7222	0,5039	0,7929	0,6865
SVM + GridSearch	0,4048	0,7556	0,5271	0,8026	0,7012
<i>Redes recurrentes</i>					
BiLSTM v1 (2 capas)	0,4277	0,7556	0,5462	0,8172	0,7158
BiLSTM Villani (1 capa)	0,4539	0,7111	0,5541	0,8333	0,7258
<i>Transformers genéricos</i>					
BERT (Configuración A)	0,3838	0,7889	0,5164	0,7848	0,6890
BERT (Configuración B)	—	—	0,5145	0,7832	0,6875
RoBERTa	—	—	0,5323	0,8123	0,7074
DeBERTa v3	—	—	0,5193	0,7783	0,6876
BERT + BT (300 frases)	0,3817	0,7889	0,5145	0,7832	0,6875
<i>Transformers de dominio</i>					
CTI-BERT	—	—	0,5147	0,7589	0,6771
SecureBERT 2.0	—	—	0,5515	0,8026	0,7125
SecBERT	—	—	0,4718	0,7427	0,6509
SecureBERT + BT (2.283)	—	—	0,5430	0,7767	0,6976
<i>Modelos con prompting (sobre 50 frases del test por restricciones computacionales*)</i>					
BART-MNLI (zero-shot)	0,1429	1,0000	0,2500	0,1600	0,1477
FLAN-T5 (2-shot)	—	—	—	—	—

* La evaluación completa sobre las 618 frases del test no fue viable con BART-large en el entorno disponible (GPU T4, 16 GB de VRAM) por el tiempo de inferencia. Véase la Sección 7.5.

TABLA 7.1. RESULTADOS SOBRE EL CONJUNTO DE TEST
OFICIAL DE SEMEVAL-2018 TASK 8 SUBTASK 1. PRECISIÓN Y
RECALL CORRESPONDEN A LA CLASE RELEVANTE.

Los guiones en las columnas de precisión y recall indican que para esos modelos solo se registraron las métricas globales (F1, accuracy y Macro F1), sin desglosar por clase.

Modelos clásicos (SVM). El SVM con TF-IDF de unigramas y bigramas alcanzó un F1 de 0,5039 sobre la clase relevante. Este resultado se utiliza como referencia inicial para valorar si los modelos posteriores aportan una mejora real. La precisión fue de 0,3869 y el *recall* de 0,7222. Esto indica que el modelo generó un número considerable de falsos positivos, pero fue capaz de identificar una parte importante de las frases relevantes del conjunto de test. El SVM con la configuración óptima seleccionada mediante búsqueda exhaustiva de hiperparámetros (descrita en la Sección 6.2) eleva el F1 hasta 0,5271. La configuración óptima conserva los bigramas, fija $C = 1,0$ y descarta el ajuste `class_weight=balanced` pese a trabajar con un dataset fuertemente desbalanceado. La reponderación por clases no mejoró el rendimiento. La explicación más probable es

que el mayor peso asignado a la clase minoritaria aumentara el *recall*, pero redujera la precisión. Al combinar ambas métricas, el F1 terminó penalizando ese desequilibrio. La versión optimizada mejora a la vez precisión (0,4048) y recall (0,7556) frente al baseline, lo que indica una mejor clasificación.

Redes recurrentes (BiLSTM). La primera versión de BiLSTM, con dos capas apiladas y embeddings GloVe congelados, obtiene un F1 de 0,5462. Ya supera al baseline SVM y se aproxima a las cifras del mejor sistema de la competición original. La segunda versión, que replica con mayor fidelidad la propuesta de Villani [5] (una sola capa BiLSTM con 256 unidades por dirección, embeddings concatenados de GloVe y trigramas de caracteres, atención de Luong y RMSprop), alcanza un F1 de 0,5541 con una exactitud del 83,33 %.

Es el mejor resultado de todo el conjunto de experimentos, por encima de cualquier Transformer (genérico o de dominio) y muy próximo al 0,5714 del paper original. La diferencia de unos dos puntos respecto al sistema ganador puede deberse a varios factores: el preentrenamiento de los embeddings, las semillas aleatorias, la configuración exacta del sobremuestreo aplicado por Villani o pequeños matices del código no reproducibles a partir del paper. Más que la diferencia exacta con el sistema original, el resultado muestra que una arquitectura recurrente relativamente pequeña puede competir con modelos preentrenados de muchos más parámetros.

Como observación adicional, la BiLSTM Villani sacrifica algo de recall (0,7111) frente a la BiLSTM v1 (0,7556) pero gana en precisión (0,4539 frente a 0,4277). La BiLSTM con atención de Luong y embeddings enriquecidos discrimina mejor, no es solo más generosa.

Transformers genéricos. BERT se evaluó bajo dos configuraciones con hiperparámetros idénticos ($lr = 5 \times 10^{-5}$, 3 épocas, lote 8) pero distinto corpus de entrenamiento. La Configuración A, entrenada sobre la fusión train+dev (10.648 frases), obtiene un F1 de 0,5164. La Configuración B, entrenada solo sobre el split oficial (9.435 frases), obtiene 0,5145. La diferencia de menos de dos décimas indica que el volumen adicional que aporta dev apenas tiene efecto en este corpus, probablemente porque el conjunto de desarrollo añade muy pocas frases relevantes (79 de las 1.213 frases de dev, frente a las 2.204 ya presentes en train).

Como los hiperparámetros son idénticos en ambas configuraciones, la diferencia de F1 refleja exclusivamente el efecto del volumen de datos de entrenamiento.

RoBERTa marca el mejor resultado de los Transformers genéricos con un F1 de 0,5323. La eliminación de la tarea NSP, el uso de máscaras dinámicas y el preentrenamiento con un volumen sustancialmente mayor de texto [8] le proporcionan una ventaja consistente sobre BERT bajo el mismo protocolo de fine-tuning, que se traduce en un F1 cerca de dos puntos superior y una exactitud del 81,23 %. DeBERTa v3 se queda en

0,5193, algo por encima de BERT pero por debajo de RoBERTa. Sus mejoras técnicas como la atención desenredada o el codificador de máscara mejorado están pensadas para tareas de comprensión donde las relaciones posicionales finas son determinantes; en una clasificación binaria sobre frases cortas como esta las mejoras arquitectónicas aportan más bien poco.

En cualquier caso, la precisión de todos los modelos supervisados sigue siendo baja en términos absolutos: ninguno supera el valor de 0,46. Esto no apunta a un fallo aislado de un modelo concreto, sino a una dificultad propia del conjunto de test. Hay 528 frases no relevantes frente a 90 relevantes, con un ratio de 5,87:1. En una distribución así, basta con clasificar erróneamente una parte pequeña de la clase mayoritaria para que la precisión de la clase relevante se vea afectada de forma notable.

Transformers de dominio. En los modelos que han sido preentrenados con corpus de ciberseguridad aparecen unas lecturas interesantes. SecureBERT 2.0 logra un F1 de 0,5515, el mejor de toda la familia Transformer y prácticamente igual al baseline SVM del paper original (0,5517). Su corpus ampliado, que además de incluir informes CTI añade código fuente, junto con la arquitectura ModernBERT [10], parece capturar bien las construcciones técnicas características de los textos de malware.

CTI-BERT obtiene un F1 de 0,5147, prácticamente idéntico al de BERT genérico (0,5164 en Configuración A, 0,5145 en Configuración B). Esto contradice la hipótesis de que un modelo entrenado de forma específica sobre informes de inteligencia de amenazas rendirá mejor en una tarea de clasificación de informes de malware. Una respuesta a ello se puede deber a que el corpus de preentrenamiento de CTI-BERT [9] se centra en informes CTI con un estilo y formato concretos que no tienen por qué coincidir con la diversidad de los informes APT de MalwareTextDB.

El peor de todos es SecBERT, con un F1 de 0,4718. El hecho de haber sido preentrenado desde cero en lugar de partir de un modelo ya consolidado como BERT, RoBERTa o SecureBERT 2.0 y tener un corpus presumiblemente menor parece pasarle factura.

La conclusión de los Transformers de dominio es que el preentrenamiento específico no garantiza por sí mejores resultados. Lo que distingue a SecureBERT 2.0 de CTI-BERT y SecBERT no es su fecha de publicación, sino tres factores concretos: parte de una arquitectura base más eficiente (ModernBERT frente a BERT estándar), se preentrenó con un corpus más diverso que combina texto de ciberseguridad y código fuente, y se beneficia de avances recientes en técnicas de preentrenamiento. La combinación de estos tres elementos es la que probablemente explica su ventaja sobre los otros dos modelos de dominio.

Aumento de datos. La técnica de back-translation se aplicó en dos escenarios distintos. Sobre BERT, partiendo de la Configuración A, se añadieron 300 frases relevantes traducidas EN→FR→EN. El F1 pasa de 0,5164 a 0,5145, una variación negativa de menos

de dos décimas que se sitúa dentro del margen de ruido atribuible a la semilla aleatoria. El aumento no produjo mejora apreciable en este caso, lo que sugiere que 300 ejemplos sintéticos resultan insuficientes para modificar el comportamiento de un modelo ya entrenado con los 10.648 ejemplos disponibles.

El segundo experimento es más ambicioso: se aplicó back-translation a las 2.283 frases relevantes del conjunto de entrenamiento, duplicando así la clase minoritaria, y se entrenó sobre ehsanaghaei/SecureBERT. El F1 resultante es de 0,5430, y aunque es peor que el 0,5515 de SecureBERT 2.0 sin aumento, muestra que la comparación tiene un matiz importante: ehsanaghaei/SecureBERT (el SecureBERT original basado en RoBERTa [43]) y cisco-ai/SecureBERT2.0-base son modelos base distintos. Lo que se está comparando no es el efecto puro de la back-translation, sino dos configuraciones que difieren en el modelo de partida y en el aumento de datos al mismo tiempo.

Por tanto, estos resultados deben interpretarse con cautela. En BERT no se observa efecto apreciable de la back-translation: el F1 apenas varía entre ambas configuraciones, lo que sugiere que 300 ejemplos sintéticos son insuficientes para mover el rendimiento de un modelo ya ajustado con todos los datos disponibles. En cambio, en el caso de SecureBERT la comparación no permite aislar de forma directa el impacto del aumento de datos, porque intervienen dos factores a la vez: el cambio de arquitectura/modelo base y la aplicación de back-translation. Aun así, el experimento resulta útil, pues muestra que ampliar de forma extensiva la clase minoritaria no garantiza una mejora cuando se trabaja con modelos ya especializados en el dominio de aplicación. Esto sugiere que, en estos modelos, el conocimiento adquirido durante el preentrenamiento puede ser más importante que una simple estrategia de duplicación de ejemplos relevantes.

Clasificación sin fine-tuning. BART-MNLI y FLAN-T5 se evaluaron como alternativas al fine-tuning supervisado. En estos enfoques, el modelo no se entrena de nuevo con el corpus de la tarea, sino que recibe una instrucción en lenguaje natural y debe clasificar la frase a partir de esa formulación. En la práctica, los resultados fueron limitados. BART-MNLI obtuvo un F1 de 0,2500 sobre una submuestra de 50 frases del conjunto de test, ya que la inferencia con BART-large sobre el conjunto completo superaba el tiempo de GPU disponible en una sesión. La precisión fue de 0,1429 y el *recall* de 1,0000. Esto indica que el modelo tendió a clasificar la mayoría de frases como relevantes, generando muchos falsos positivos. La accuracy del 16 % confirma que, con esta formulación, BART-MNLI no consiguió separar adecuadamente las dos clases.

FLAN-T5 no se evaluó cuantitativamente sobre el conjunto de test. Las restricciones de memoria y tiempo de inferencia en el entorno disponible impidieron procesar las 618 frases en una sesión. Por este motivo no se incluye en la tabla de resultados.

En bruto, estos números parecen indicar que el prompting no obtiene buenos resultados para esta tarea, pero esta lectura sería precipitada. La limitación real del experimento es el hardware: en una GPU T4 con 16 GB de VRAM no caben los modelos abiertos

de gran tamaño (Llama-3, Mixtral) ni siquiera bajo cuantización agresiva, y los modelos propietarios de gran escala requieren API externas. Existen alternativas de acceso gratuito, como los servicios de inferencia de Groq para modelos abiertos (LLaMA-3, Mixtral), que podrían permitir una evaluación más justa del prompting sin coste de infraestructura. Esta exploración queda pendiente como trabajo futuro. BART-large tiene 400 millones de parámetros y FLAN-T5-base apenas 250 millones; ambos están órdenes de magnitud por debajo de los LLMs que sí han mostrado capacidad de clasificación zero-shot competitiva en tareas similares [44], [51]. Los resultados no invalidan el prompting, solo indican que con los modelos accesibles en este entorno el fine-tuning supervisado es superior.

7.3. Análisis de errores

Para entender mejor los límites de los modelos evaluados, en esta sección se hace un análisis cualitativo de los errores cometidos sobre el conjunto de test. Dado que la métrica principal es el F1-score de la clase minoritaria, el análisis se centra en dos tipos de error: los **falsos positivos** (frases predichas como relevantes que en realidad no lo son) y los **falsos negativos** (frases relevantes que el modelo clasifica como no relevantes). Ambos penalizan el F1 desde perspectivas opuestas: los falsos positivos reducen la precisión, mientras que los falsos negativos reducen el recall.

7.3.1. Matriz de confusión del mejor modelo

Para concretar el perfil de errores del modelo con mejor F1, la Tabla 7.2 muestra su matriz de confusión sobre las 618 frases del conjunto de test.

	Predicho: no relevante	Predicho: relevante
Real: no relevante	451 (TN)	77 (FP)
Real: relevante	26 (FN)	64 (TP)

TABLA 7.2. MATRIZ DE CONFUSIÓN DE LA BILSTM
(REPLICACIÓN DE VILLANI) SOBRE EL CONJUNTO DE TEST

De las 90 frases relevantes, el modelo identifica 64 (recall 0,7111) y deja escapar 26. De las 528 no relevantes, clasifica mal 77 como relevantes, lo que explica la precisión de 0,4539. El equilibrio entre ambos tipos de error es la razón de que este modelo obtenga el F1 más alto del conjunto.

La Tabla 7.1 permite ver diferencias claras en el tipo de error que comete cada familia de modelos. En los modelos con mucho *recall* y poca precisión, como BERT en la Configuración A ($P = 0,3838$ y $R = 0,7889$), el problema principal son los falsos positivos. Es decir, el modelo detecta muchas frases relevantes, pero también marca como relevantes bastantes frases que no lo son según la anotación oficial.

La replicación de Villani presenta un equilibrio algo mejor entre ambos errores. Con una precisión de 0,4539 y un *recall* de 0,7111, no alcanza el mayor *recall* del conjunto, pero reduce parte de los falsos positivos respecto a modelos más sensibles como BERT. Ese equilibrio explica que obtenga el F1 más alto.

7.3.2. Tipos de error más frecuentes

En las predicciones incorrectas se han identificado tres tipos de error más frecuentes, independientemente del modelo empleado.

Falsos positivos asociados a vocabulario técnico. Una parte de los falsos positivos aparece en frases que contienen términos propios de ciberseguridad, pero que no describen una acción concreta del malware.

“The report describes several known malware families, including ransomware and trojans.”

“Figure 2 shows the infection chain used by the threat actor.”

Estas frases contienen términos como *malware*, *ransomware* o *infection* que los modelos asocian fuertemente con la clase relevante. Sin embargo, ninguna describe una acción concreta del malware: la primera simplemente menciona familias y la segunda hace referencia a una figura. Los modelos basados en TF-IDF son especialmente vulnerables a este tipo de error, ya que su representación no captura si el vocabulario técnico aparece en un contexto descriptivo o en uno funcional. Los Transformers, pese a su mayor capacidad contextual, también incurrir en este error con frases introductoras o de contextualización que comparten vocabulario con las frases verdaderamente relevantes.

Falsos negativos en frases con acción poco explícita. Otro grupo de falsos negativos aparece en frases que sí describen un comportamiento relevante, pero lo hacen de forma indirecta o con un vocabulario menos característico de los informes técnicos. En estos casos, la acción del malware no siempre queda expresada mediante términos claramente asociados a ejecución, modificación, conexión o descarga. Por ello, el modelo puede no reconocer la frase como relevante aunque la anotación oficial sí la considere positiva. Algunos ejemplos son:

“Once the victim opens the attachment, the attacker gains access to the system.”

“The file was designed to steal sensitive information from infected machines.”

Estas frases son relevantes según el esquema de anotación MAEC (*Malware Attribute Enumeration and Characterization*) [56], porque describen acciones del atacante o

del malware. El problema es que no siempre usan marcadores léxicos muy reconocibles, como *encrypts*, *drops* o *connects*. Por ello, los modelos pueden tener más dificultad para asociarlas con la clase relevante.

Casos ambiguos en la frontera de clases. El tercer tipo de error aparece en frases cuya etiqueta no es evidente incluso desde una lectura humana. Como se explicó en el Capítulo 5, algunas frases se sitúan cerca del límite entre las clases relevante y no relevante. En estos casos, la decisión depende mucho del contexto y del criterio de anotación aplicado. Un ejemplo representativo es:

“The attackers used spear-phishing emails to gain initial access to the target network.”

Esta frase describe una técnica de ataque, en concreto un posible vector de infección inicial, pero no explica qué hace después el malware dentro del sistema. Por eso, su etiqueta puede depender de cómo se interprete el criterio de relevancia del esquema MAEC (*Malware Attribute Enumeration and Characterization*) [56].

En casos como este, distintos modelos pueden tomar decisiones diferentes ante la misma frase. Esto no apunta necesariamente a un fallo claro del modelo, sino a la propia ambigüedad del ejemplo. La frase contiene información relacionada con la amenaza, pero no siempre encaja de forma directa con la definición de comportamiento relevante. Estos casos muestran una limitación práctica de la tarea: para reducir este tipo de errores haría falta un criterio de anotación más detallado o un esquema de etiquetas más granular.

7.3.3. Diferencias entre familias de modelos

El perfil de errores varía de forma apreciable entre las distintas familias de modelos evaluadas.

El **SVM con TF-IDF** tiende a fallar en frases con vocabulario poco técnico o con construcciones pasivas, donde la relevancia no aparece asociada a verbos de acción directos. En la variante básica, el *recall* es de 0,7222, por debajo de modelos más sensibles como BERT en la Configuración A (*recall* de 0,7889, descrita en la Sección 6.1). Sus falsos positivos, en cambio, suelen estar asociados a frases que mencionan nombres de malware, familias de amenazas o términos técnicos, pero en un contexto más descriptivo que operativo.

SecureBERT 2.0 obtiene un resultado más equilibrado dentro de los modelos Transformer, con un F1 de 0,5515. Su preentrenamiento con textos de CTI (*Cyber Threat Intelligence*) y código fuente puede haber favorecido una mejor representación de construcciones técnicas propias de informes de malware. Aun así, al no disponer de precisión y *recall* desglosados para este modelo, no es posible confirmar con detalle qué proporción de falsos positivos y falsos negativos produce.

BART-MNLI, evaluado en modo *zero-shot* sobre 50 frases del test, muestra el comportamiento más extremo. Con un *recall* de 1,0000 y una precisión de 0,1429, el modelo tendió a clasificar la mayoría de frases como relevantes. Este resultado es coherente con una formulación de la tarea que no estuvo ajustada al dominio específico del malware: las hipótesis usadas en la inferencia textual no capturaron con suficiente precisión qué debía considerarse una frase relevante.

Perfil de precisión y recall por familia. Los resultados de la Tabla 7.1 permiten comparar el tipo de error predominante en cada familia. Los modelos con mayor recall y menor precisión, como BERT en la Configuración A ($P=0,3838$, $R=0,7889$), tienden a generar muchos falsos positivos: detectan la mayoría de frases relevantes pero también marcan como relevantes frases que no lo son. La BiLSTM Villani presenta el equilibrio más favorable ($P=0,4539$, $R=0,7111$), lo que explica su F1 más alto. En cualquier caso, la precisión de todos los modelos supervisados es baja en términos absolutos: ninguno supera 0,46. Esto no apunta a un fallo aislado, sino a la dificultad inherente del conjunto de test, con un ratio de 5,87:1 entre clases.

7.4. Discusión

Ordenando los modelos supervisados por F1 de la clase relevante se identifica una jerarquía clara (Tabla 7.3). La replicación de Villani (0,5541) encabeza el ranking, seguida de SecureBERT 2.0 (0,5515) y BiLSTM v1 (0,5462). A continuación aparece SecureBERT + BT (0,5430), RoBERTa (0,5323) y el SVM optimizado (0,5271). En la zona media se agrupan BERT Configuración A (0,5164), DeBERTa v3 (0,5193), CTI-BERT (0,5147), BERT + BT (0,5145) y BERT Configuración B (0,5145). El SVM básico (0,5039) y SecBERT (0,4718) cierran la tabla entre los modelos supervisados.

Pos.	Modelo	F1
1	BiLSTM Villani (replicación)	0,5541
2	SecureBERT 2.0	0,5515
3	BiLSTM v1	0,5462
4	SecureBERT + BT	0,5430
5	RoBERTa	0,5323
6	SVM + GridSearch	0,5271
7	DeBERTa v3	0,5193
8	BERT (Config. A)	0,5164
9	CTI-BERT	0,5147
10	BERT + BT	0,5145
10	BERT (Config. B)	0,5145
11	SVM baseline	0,5039
12	SecBERT	0,4718

TABLA 7.3. RANKING DE MODELOS SUPERVISADOS POR F1 DE LA CLASE RELEVANTE

Los valores de F1 se concentran en una franja relativamente estrecha, entre 0,47 y 0,55. Esto sugiere un techo práctico marcado por la propia dificultad de la tarea: los casos ambiguos descritos en el Capítulo 5 no admiten un clasificador perfecto, y el desbalanceo entre particiones complica aún más la generalización. Dado además que los experimentos se realizaron con una única semilla aleatoria, las diferencias menores de uno o dos puntos de F1 deben interpretarse con cautela.

Para situar estos resultados en el contexto de la competición original, la Tabla 7.4 compara los mejores modelos de este trabajo con los sistemas presentados en SemEval-2018 Task 8 SubTask 1.

Origen	Modelo	F1
<i>Sistemas de la competición original</i>		
Competición	Villani (ganador)	0,5714
Competición	Flytxt_NTNU	0,5517
Competición	Baseline SVM (organizadores)	0,5517
Competición	NLP Foundation	0,5130
Competición	HCCL	0,4904
Competición	TeamDL	0,4871
Competición	UMBC	0,4561
Competición	Digital Operatives	0,4213
<i>Mejores resultados de este trabajo</i>		
Este TFG	BiLSTM Villani (replicación)	0,5541
Este TFG	SecureBERT 2.0	0,5515
Este TFG	BiLSTM v1	0,5462
Este TFG	RoBERTa	0,5323
Este TFG	SVM + GridSearch	0,5271

TABLA 7.4. COMPARATIVA ENTRE LOS MEJORES MODELOS DE ESTE TRABAJO Y LOS SISTEMAS PRESENTADOS EN LA COMPETICIÓN ORIGINAL

La replicación de Villani queda 1,7 puntos por debajo del sistema original, una distancia razonable teniendo en cuenta que el paper no proporciona todos los detalles de configuración como los tamaños exactos del sobremuestreo, valores de las semillas o la configuración precisa de la regularización.

Resulta más informativo comparar contra el resto del podio. SecureBERT 2.0 (0,5515) se sitúa prácticamente al mismo nivel que el baseline SVM original y que Flytxt_NTNU (ambos en 0,5517), y apenas a dos puntos de Villani. La BiLSTM v1 (0,5462) supera a todos los equipos de la competición salvo a los tres primeros. RoBERTa (0,5323), el SVM optimizado (0,5271), BERT en cualquiera de sus configuraciones, DeBERTa, CTI-BERT y la propia BiLSTM v1 quedan todos por encima del cuarto clasificado de la competición original (NLP Foundation, 0,5130). SecBERT (0,4718) es el único modelo supervisado de este trabajo que se sitúa por debajo del cuarto clasificado original, aunque aún por encima de UMBC y Digital Operatives.

Visto desde otra perspectiva: si los resultados de este trabajo se hubieran presentado en su día a SemEval-2018, la BiLSTM Villani replicada habría ocupado aproximadamente la tercera posición, SecureBERT 2.0 estaría empatado con Flytxt_NTNU y el baseline original, y todos los demás Transformers entrarían en la mitad superior de la clasificación. Que un modelo de 2025 (SecureBERT 2.0) iguale al baseline de 2018 sin superarlo con creces confirma la dificultad intrínseca de la tarea: no se trata de un problema donde el paso del tiempo y la mejora de los modelos base resuelvan la cuestión.

7.5. Limitaciones del estudio

Antes de cerrar el capítulo conviene explicitar las limitaciones del estudio. Casi todas tienen un origen común: toda la experimentación se ejecutó en la versión gratuita de Google Colab, que impone tres restricciones difíciles de sortear. La primera es la VRAM disponible (16-GB en la GPU T4, ocasionalmente una P100), insuficiente para cargar LLMs modernos incluso con cuantización. La segunda es el cupo diario de GPU, que se agota tras unas horas de uso intensivo y bloquea el acceso durante el resto del día. La tercera son las desconexiones automáticas, que interrumpen entrenamientos largos y obligan a relanzarlos desde cero. Las decisiones que se describen a continuación están condicionadas por este marco, no son fruto del descuido metodológico.

Submuestra en la evaluación del prompting. BART-MNLI se evaluó sobre 50 frases del test, no sobre las 618 completas. La inferencia zero-shot con BART-large es órdenes de magnitud más lenta que el fine-tuning supervisado y procesar el test entero superaba el cupo de GPU asignado en una sesión. FLAN-T5 ni siquiera dispone de evaluación cuantitativa formal por el mismo motivo. Es por ello que las cifras del prompting deben leerse como orientativas.

Único protocolo de hiperparámetros por modelo. Una búsqueda en malla sería sobre los seis Transformers evaluados habría exigido decenas de entrenamientos adicionales. A 30–45 minutos por configuración, con el cupo diario agotándose tras tres o cuatro ejecuciones, hace inviable esa exploración. Es por ello que solo se hizo una búsqueda exhaustiva en el SVM (cuyo entrenamiento dura segundos, no minutos) y se fijó una configuración estándar razonable para los Transformers. El ranking entre modelos próximos en F1 podría variar con más exploración.

Una sola semilla aleatoria por experimento. En el rango de F1 observado (0.47-0.55), la varianza entre semillas puede ser de varias décimas. Repetir cada experimento con tres o cinco semillas distintas habría triplicado o quintuplicado el consumo de GPU, algo imposible bajo las restricciones del entorno. Las comparaciones más finas (diferencias por debajo de un punto de F1) hay que tomarlas con la cautela que esto impone.

Back-translation con un solo idioma intermedio. Se usó francés (Helsinki-NLP/opus-mt-en-fr) por su disponibilidad y velocidad. Probar varios idiomas (alemán, español, chino) o combinarlos habría enriquecido el aumento, pero cada idioma adicional implicaba descargar otro modelo MarianMT (cientos de MB), procesar las 2.283 frases relevantes dos veces y reentrenar el clasificador. Como sugiere parte de la literatura sobre aumento textual [52], multi-idioma puede mejorar resultados, pero queda como línea futura.

Comparabilidad entre modelos. Como se detalla en la Sección 6.1, todos los modelos supervisados se entrenaron sobre la fusión train+dev (10.648 frases), con la única excepción de BERT Configuración B, que utilizó únicamente el split oficial (9.435 frases) como punto de comparación deliberado. La diferencia de F1 entre ambas configuraciones de BERT fue de menos de dos décimas (0,5164 frente a 0,5145), lo que confirma que el volumen adicional de dev tiene un impacto prácticamente nulo en este corpus y que los resultados del conjunto son comparables entre sí con independencia de esta distinción.

Estas limitaciones no invalidan los resultados, pero sí acotan su alcance. Los hallazgos sobre la competitividad de la BiLSTM, la efectividad parcial del preentrenamiento de dominio y la ausencia de mejora apreciable de la back-translation sobre BERT en este corpus son sólidos dentro del marco experimental descrito. En el Capítulo 9 se proponen líneas de trabajo futuro orientadas a superar estas restricciones, en su mayoría mediante acceso a infraestructura computacional más potente.

8. MARCO REGULADOR, IMPACTO SOCIOECONÓMICO Y SOSTENIBILIDAD

En este capítulo se examina el contexto legal, económico y ético en el que se inscribe este trabajo. Se analizan las normativas vigentes en España y la Unión Europea que afectan al desarrollo de sistemas de inteligencia artificial aplicados a la ciberseguridad, las licencias de los recursos empleados, el impacto socioeconómico de la automatización del análisis de amenazas y la contribución del trabajo a los Objetivos de Desarrollo Sostenible de las Naciones Unidas.

8.1. Marco regulador

8.1.1. Normativa europea de ciberseguridad

La ciberseguridad es uno de los ámbitos donde la Unión Europea ha desarrollado un marco normativo más activo en los últimos años. La legislación más relevante para el contexto de este trabajo es la **Directiva NIS2** (Directiva (UE) 2022/2555, de 14 de diciembre de 2022) [68], que actualiza y deroga la anterior Directiva NIS de 2016. La NIS2 proporciona medidas legales para impulsar el nivel general de ciberseguridad en la UE y la resiliencia de las infraestructuras críticas y de los servicios digitales en Europa, estableciendo obligaciones de ciberseguridad, de supervisión y ejecución para los Estados miembros, medidas para la gestión de riesgos y obligaciones de notificación para las entidades en su ámbito de aplicación [2], [68].

En España, la transposición de la NIS2 aún está en proceso. El Consejo de Ministros aprobó en enero de 2025 un Anteproyecto de Ley de Coordinación y Gobernanza de la Ciberseguridad, cuya entrada en vigor se estima a lo largo de 2026. Aunque este trabajo es de naturaleza investigadora y no implica el despliegue de un sistema real en sectores críticos, un hipotético sistema de clasificación de amenazas integrado en una plataforma CTI operativa podría quedar dentro del ámbito de la NIS2 si la entidad que lo adoptara estuviera sujeta a la directiva. En ese caso, el sistema debería someterse a las medidas de gestión de riesgos y a las obligaciones de notificación de incidentes que establece el Capítulo IV de la directiva.

8.1.2. Reglamento General de Protección de Datos y LOPDGDD

El **Reglamento General de Protección de Datos** (RGPD) [69] regula en la Unión Europea el tratamiento de datos personales. En España se complementa con la **Ley Orgánica 3/2018, de Protección de Datos Personales y Garantía de los Derechos Digitales**

(LOPDGDD) [70], que adapta el marco europeo al ordenamiento jurídico español e incorpora garantías adicionales sobre derechos digitales.

En este trabajo, el dataset utilizado procede de MalwareTextDB [4] y del repositorio asociado a SemEval-2018 Task 8 [3]. Se trata de informes técnicos de malware obtenidos de fuentes públicas de inteligencia de amenazas. El contenido analizado describe programas maliciosos, técnicas de ataque, artefactos del sistema y comportamientos observables, no información personal sobre individuos. Por este motivo, el uso del corpus en el contexto experimental del trabajo no plantea, en principio, un tratamiento directo de datos personales.

Aun así, esta valoración cambiaría en un despliegue real. Si el sistema procesara informes internos de incidentes, podría recibir información relativa a empleados, clientes, direcciones IP, nombres de equipo, cuentas de usuario u otros datos vinculados a una organización concreta. En ese escenario sería necesario aplicar los principios del RGPD, especialmente la minimización de datos, la limitación de la finalidad y la integridad y confidencialidad del tratamiento, recogidos en el artículo 5 del reglamento [69].

8.1.3. Ley de Inteligencia Artificial de la Unión Europea (*AI Act*)

En agosto de 2024 entró en vigor el **Reglamento (UE) 2024/1689, conocido como *AI Act*** [71], la primera ley integral sobre inteligencia artificial a nivel mundial. Su característica principal es la clasificación de los sistemas de IA en cuatro niveles de riesgo: inaceptable, alto, limitado y mínimo, estableciendo obligaciones proporcionales a cada nivel.

Un sistema de clasificación automática de frases de malware, en su uso investigador actual, correspondería previsiblemente a la categoría de **riesgo mínimo o limitado**, ya que no toma decisiones autónomas que afecten directamente a derechos fundamentales ni opera en sectores de alta criticidad como la sanidad o la justicia. Sin embargo, si el sistema fuera desplegado como componente de un sistema de inteligencia de amenazas que informara decisiones de respuesta a incidentes en infraestructuras críticas, podría requerir una evaluación de conformidad más rigurosa, incluyendo documentación técnica detallada, pruebas de robustez y mecanismos de supervisión humana, tal como exige el *AI Act* [71] para sistemas de alto riesgo.

Las obligaciones que en ese caso serían aplicables incluirían la evaluación de riesgos previa al despliegue, la trazabilidad de los datos de entrenamiento, la transparencia sobre las capacidades y limitaciones del sistema, y la garantía de que un analista humano puede revisar y revertir cualquier decisión tomada a partir de sus predicciones.

8.1.4. Propiedad intelectual y licencias

El desarrollo de este trabajo ha utilizado recursos con distintas licencias de software y de datos. Las más relevantes son las siguientes:

- **Python Software Foundation License (PSFL)**: licencia asociada al lenguaje Python, utilizado como base del desarrollo experimental.
- **Apache 2.0**: licencia bajo la que se distribuye la librería *Transformers* de HuggingFace [14]. También se ha tenido en cuenta para los modelos SecureBERT [43] y SecureBERT 2.0 [10], de acuerdo con la información disponible en sus repositorios.
- **MIT**: licencia presente en varias utilidades empleadas durante el preprocesamiento y en componentes asociados al ecosistema de HuggingFace.
- **BSD**: licencia utilizada por herramientas como *scikit-learn* [12], PyTorch [13], NumPy y Pandas.
- **Uso académico y de investigación**: MalwareTextDB [4] y los datos de SemEval-2018 Task 8 [3] se encuentran disponibles públicamente para fines de investigación. El dataset preprocesado generado en este trabajo y publicado en HuggingFace⁸ mantiene esa orientación académica, no comercial y con atribución a las fuentes originales.
- **Licencia propietaria de NVIDIA**: aplicable a CUDA y cuDNN, utilizados para acelerar el entrenamiento de modelos en GPU dentro del entorno de Google Colab.

El código fuente desarrollado durante el trabajo se ha publicado en el repositorio oficial del proyecto⁹. Esta publicación facilita la consulta, reproducción y posible extensión de los experimentos con fines académicos.

8.1.5. Consideraciones éticas y de uso dual

El uso de técnicas de Procesamiento del Lenguaje Natural (PLN) para analizar informes de malware presenta una cuestión ética relevante: el posible **uso dual** de la tecnología. Un sistema capaz de localizar automáticamente frases que describen comportamientos de malware puede tener aplicaciones defensivas, como apoyar la respuesta a incidentes o facilitar la extracción de inteligencia accionable. Sin embargo, también podría utilizarse de forma indebida para identificar técnicas de ataque descritas en informes públicos y reutilizarlas con fines ofensivos.

⁸<https://huggingface.co/datasets/hugobecerra/DATASET3.0>

⁹<https://github.com/100495717/TFG-HUGO-BECERRA>

Este riesgo no es exclusivo de este trabajo, sino común a muchas herramientas de ciberseguridad. Por ello, el problema no está en investigar estas técnicas, sino en definir adecuadamente su contexto de uso, su distribución y sus posibles limitaciones de acceso [1].

En este trabajo, los modelos se entrenan únicamente con datos públicos y con una finalidad académica. La publicación del código y de los recursos generados busca favorecer la reproducibilidad de los experimentos y permitir que otros investigadores puedan revisar, comparar o ampliar los resultados obtenidos.

8.2. Impacto socioeconómico

Los modelos evaluados en este trabajo se sitúan en un ámbito con impacto social directo: el análisis de amenazas de ciberseguridad. Las organizaciones públicas y privadas trabajan con un volumen creciente de informes de malware, boletines de vulnerabilidades e informes de inteligencia de amenazas [1]. En ese contexto, automatizar una primera fase de filtrado, como la clasificación de frases relevantes y no relevantes, puede ayudar a reducir parte del trabajo manual. El objetivo no sería sustituir al analista, sino facilitar que dedique más tiempo a tareas que requieren criterio experto, como la priorización de incidentes, la interpretación técnica de evidencias o la relación entre campañas, herramientas y grupos de amenazas.

Desde el punto de vista económico, el trabajo se enmarca en un sector en crecimiento. Según el Estudio sobre la industria de la ciberseguridad en España 2025, elaborado por el Instituto Nacional de Ciberseguridad (INCIBE) y CONETIC, la industria alcanzó en 2024 una facturación de 6.351 millones de euros, equivalente al 4,65 % del sector de Tecnologías de la Información y las Comunicaciones (TIC) en España [72]. El mismo informe señala que el empleo en el sector creció un 35,14 % entre 2021 y 2025, hasta superar los 164.000 profesionales, con una previsión de 282.000 en 2029 [72]. Además, el mercado español de productos y servicios de ciberseguridad creció un 14,2 % en 2024, hasta alcanzar los 2.500 millones de euros, con una previsión de crecimiento medio anual del 14 % durante 2025 y 2026 [73]. En este escenario, las herramientas de Procesamiento del Lenguaje Natural (PLN) aplicadas a inteligencia de amenazas pueden contribuir a mejorar la eficiencia de un sector con alta demanda de perfiles especializados.

Desde una perspectiva social más amplia, este tipo de sistemas también puede facilitar el acceso a capacidades de análisis que no siempre están disponibles para organizaciones pequeñas. Una pyme o una entidad pública con recursos limitados puede tener dificultades para mantener un equipo amplio de analistas de ciberseguridad. Un sistema que ayude a filtrar informes de amenazas y a localizar la información más relevante podría reducir esa barrera inicial y permitir un mejor aprovechamiento de fuentes públicas de inteligencia de amenazas [2]. Esta utilidad debe entenderse como apoyo al análisis, no como sustitución de la revisión experta.

También debe considerarse el impacto ambiental asociado al uso de modelos de inteligencia artificial. Las arquitecturas Transformer de gran tamaño pueden requerir un consumo energético elevado durante su entrenamiento y despliegue [74]. Por ello, resulta conveniente priorizar enfoques más eficientes cuando el contexto lo permita, como el fine-tuning sobre modelos ya preentrenados, la reutilización de modelos existentes, la destilación o el uso de arquitecturas más pequeñas. En este trabajo, los experimentos se realizaron en Google Colab con GPU (*Graphics Processing Unit*) T4 compartidas, por lo que el impacto individual del proyecto es limitado. Aun así, este aspecto debe tenerse en cuenta si sistemas similares se desplegaran a mayor escala.

Por último, el uso de PLN para analizar informes de malware plantea cuestiones éticas relacionadas con el uso dual de la tecnología [1]. Un sistema capaz de identificar automáticamente frases sobre comportamientos de malware puede emplearse con fines defensivos, por ejemplo para apoyar la respuesta a incidentes o acelerar la extracción de inteligencia accionable. Sin embargo, también podría utilizarse de forma indebida para localizar técnicas de ataque descritas en informes públicos. Por este motivo, la publicación y el uso de este tipo de herramientas deben acompañarse de criterios de responsabilidad, supervisión humana y control del contexto de aplicación.

En este sentido, marcos como el *AI Act* [71] y la Directiva NIS2 [68] ofrecen un marco normativo relevante para orientar el uso de sistemas de inteligencia artificial en entornos de ciberseguridad. Aunque este trabajo no despliega un sistema operativo real, sí permite identificar aspectos que deberían considerarse en una posible aplicación futura: transparencia sobre las limitaciones del modelo, revisión humana de los resultados y proporcionalidad en el uso de la automatización.

8.3. Contribución a los Objetivos de Desarrollo Sostenible

Los **Objetivos de Desarrollo Sostenible** (ODS) forman parte de la Agenda 2030 de Naciones Unidas, aprobada en 2015 [75]. Estos objetivos recogen metas sociales, económicas y medioambientales orientadas a mejorar las condiciones de vida, proteger el planeta y reforzar el desarrollo sostenible. Aunque este trabajo tiene un alcance académico y experimental, puede relacionarse con varios de estos objetivos por su aplicación al análisis de amenazas de ciberseguridad y por la publicación abierta de los recursos generados.

ODS 9: Industria, Innovación e Infraestructura. El ODS 9 se centra en promover infraestructuras resilientes, impulsar la innovación y favorecer una industrialización sostenible [75]. La ciberseguridad forma parte de esa resiliencia, ya que muchos servicios digitales dependen de la capacidad de las organizaciones para prevenir, detectar y responder ante amenazas. En este contexto, el trabajo contribuye al ODS 9 mediante la evaluación de técnicas de Procesamiento del Lenguaje Natural (PLN) aplicadas al análisis automático

de informes de malware.

La contribución se produce principalmente en dos planos. Por un lado, se comparan distintas familias de modelos para una tarea concreta de inteligencia de amenazas: identificar frases relevantes dentro de informes técnicos. Por otro, la publicación del código, el dataset preprocesado y los resultados experimentales permite que otros estudiantes, investigadores o profesionales puedan revisar y reutilizar el trabajo. Esta apertura facilita la continuidad de la investigación y favorece la transferencia de conocimiento hacia un sector con demanda creciente de soluciones de análisis automatizado [72], [73].

ODS 16: Paz, Justicia e Instituciones Sólidas. El ODS 16 busca promover sociedades pacíficas e instituciones eficaces, responsables e inclusivas [75]. En el ámbito digital, la seguridad de las instituciones depende en parte de su capacidad para proteger servicios públicos, infraestructuras críticas y sistemas de información frente a ataques. La Directiva NIS2 refleja precisamente esta preocupación al reforzar las obligaciones de ciberseguridad y resiliencia para entidades esenciales e importantes [68].

Este trabajo se relaciona con el ODS 16 porque aborda una fase inicial del análisis de amenazas: la identificación automática de información relevante en informes de malware. Una herramienta de este tipo podría ayudar a equipos de seguridad a revisar documentación técnica con mayor rapidez y a priorizar información útil para la respuesta ante incidentes. Su aportación debe entenderse como apoyo al trabajo del analista, no como sustitución de la supervisión humana. Aun así, una gestión más eficiente de la inteligencia de amenazas puede contribuir a reforzar la protección de organizaciones e instituciones frente a riesgos digitales [2].

ODS 4: Educación de Calidad. El ODS 4 tiene como finalidad garantizar una educación inclusiva y de calidad, así como promover oportunidades de aprendizaje durante toda la vida [75]. La UNESCO señala que la inteligencia artificial puede apoyar la innovación educativa y contribuir al desarrollo de nuevas formas de enseñanza y aprendizaje [76].

En este sentido, el trabajo contribuye al ODS 4 por su carácter reproducible y formativo. La publicación abierta del código, del dataset preprocesado y de los resultados permite que otros estudiantes o investigadores puedan estudiar el proceso completo: preparación de datos, entrenamiento de modelos, evaluación y análisis de resultados. Además, el trabajo reúne dos áreas que no siempre aparecen juntas en materiales docentes introductorios: ciberseguridad y Procesamiento del Lenguaje Natural.

La comparación de distintas configuraciones de modelos también tiene valor educativo. Permite observar cómo cambian los resultados según la arquitectura, el preentrenamiento, el aumento de datos o la estrategia de evaluación. Por tanto, además de su aportación experimental, el trabajo puede servir como referencia práctica para comprender cómo se evalúan modelos de clasificación textual en un dominio técnico especializado.

8.4. Metodología de desarrollo y planificación

8.4.1. Metodología incremental experimental

El desarrollo de este trabajo no se ajusta a una metodología de gestión de proyectos software clásica. Marcos como Scrum o Kanban están pensados para equipos que construyen un producto con requisitos cambiantes y entregas frecuentes a un cliente [77], una situación que no corresponde a un trabajo de investigación realizado por una sola persona y orientado a comparar modelos sobre una tarea fija. Por ese motivo se adoptó una metodología **incremental experimental**, que mantiene el carácter iterativo de los enfoques ágiles pero lo adapta al objetivo del trabajo [78].

La idea central de esta metodología es que cada familia de modelos se incorpora como una nueva iteración del trabajo. En cada ciclo se implementa un modelo, se entrena, se evalúa sobre el conjunto de test con la métrica principal, el F1 de la clase relevante, y se analiza su comportamiento.

Los resultados de cada ciclo sirven para orientar los experimentos posteriores. Por ejemplo, el SVM con TF-IDF estableció una primera referencia de rendimiento frente a la que comparar modelos más complejos. Del mismo modo, el comportamiento observado en BERT motivó la prueba de aumento de datos mediante *back-translation*. Así, los experimentos no se plantean como bloques aislados, sino como una secuencia progresiva en la que cada decisión se apoya en los resultados obtenidos previamente.

8.4.2. Planificación temporal

El trabajo se organizó en diez fases. Las primeras son secuenciales, pero varias de las fases experimentales se solaparon parcialmente, ya que el análisis de un modelo podía iniciarse mientras se entrenaba el siguiente. La redacción de la memoria se mantuvo como una actividad transversal a lo largo de todo el periodo, y no como una fase final aislada. La Tabla 8.1 resume las fases, sus tareas principales, la duración estimada en semanas y el entregable asociado. Las duraciones son relativas y reflejan la dedicación aproximada a cada fase, no un calendario cerrado.

Fase	Tareas principales	Sem.
1. Revisión bibliográfica	Estudio de SemEval-2018 Task 8, PLN, Transformers y modelos de dominio	4
2. Preparación del dataset	Limpieza, normalización y partición; publicación en HuggingFace	3
3. Baseline SVM	Representación TF-IDF y búsqueda de hiperparámetros	2
4. Modelos recurrentes	BiLSTM v1 y replicación de Villani	3
5. Transformers genéricos	fine-tuning de BERT, RoBERTa y DeBERTa v3	3
6. Transformers de dominio	fine-tuning de CTI-BERT, SecureBERT 2.0 y SecBERT	2
7. Aumento de datos	Back-translation EN–FR–EN sobre BERT y SecureBERT	2
8. Prompting	Inferencia zero-shot y few-shot (BART-MNLI, FLAN-T5)	1
9. Análisis de resultados	Consolidación de métricas, ranking y análisis de errores	3
10. Redacción y revisión	Redacción de la memoria e incorporación de correcciones	6

TABLA 8.1. PLANIFICACIÓN DEL TRABAJO POR FASES

8.5. Presupuesto

A continuación se presenta una estimación del coste asociado al trabajo. No se trata de un coste real facturado, sino de un presupuesto académico orientativo. El objetivo es valorar el esfuerzo dedicado y los recursos utilizados como si el proyecto se hubiera desarrollado en un contexto profesional.

La estimación se divide en dos bloques: costes de personal y costes de infraestructura y software.

8.5.1. Coste de personal

El Real Decreto 1125/2003 [79] establece que cada crédito ECTS (*European Credit Transfer and Accumulation System*) equivale a entre 25 y 30 horas de trabajo del estudiante. En un Trabajo Fin de Grado de 12 créditos, esto supone una dedicación teórica de entre 300 y 360 horas.

En este caso, la dedicación estimada ha sido superior, aproximadamente 450 horas. Esta diferencia se debe principalmente a la carga experimental del proyecto: preparación

del dataset, entrenamiento y evaluación de quince modelos, revisión de resultados y análisis posterior.

Para estimar el coste del trabajo del estudiante se ha utilizado una tarifa orientativa de 20,71 €/h, asociada al perfil de un ingeniero informático junior. Esta cifra se ha tomado como referencia a partir de los datos salariales de la Encuesta Anual de Estructura Salarial (EAES) de 2023 [80] y de los tipos de cotización a la Seguridad Social vigentes en 2025 [81].

Concepto	Horas	Coste/hora	Total
Estudiante (ingeniero junior)	450	20,71 €	9.319,50 €
Subtotal personal			9.319,50 €

TABLA 8.2. COSTE DE PERSONAL (ESTIMACIÓN)

8.5.2. Coste de infraestructura y software

La experimentación se ejecutó en la versión gratuita de Google Colab, utilizando GPU (*Graphics Processing Unit*) T4 o P100 según disponibilidad. Como se explica en la Sección 7.5, esta versión impone restricciones de VRAM, cupo diario y posibles desconexiones, pero no supuso un coste real para el proyecto¹⁰

El software empleado también tuvo coste nulo, ya que se utilizaron herramientas de código abierto: Python, *scikit-learn* [12], PyTorch [13] y la librería *Transformers* de HuggingFace [14].

Además, se incluye una estimación de la amortización parcial del equipo personal utilizado y de los costes indirectos asociados al desarrollo del trabajo. Estos costes agrupan principalmente el consumo eléctrico, estimado a partir del precio medio de la electricidad en España [82], y la conexión a Internet [83]. El desglose se recoge en la Tabla 8.3.

Los costes indirectos se han estimado considerando un período de desarrollo de seis meses. El consumo eléctrico se ha calculado a partir de una potencia media del equipo de 65 W durante aproximadamente 4 horas diarias de trabajo, aplicando el precio medio del PVPC en España durante 2025 (0,17 €/kWh) [82], lo que resulta en un coste aproximado de 24 € en electricidad. La conexión a Internet se ha estimado a partir de una tarifa de fibra estándar de 30 €/mes [83], sumando 180 € en seis meses. El resto, hasta los 500 € totales, corresponde a la amortización de software auxiliar (antivirus, licencias de sistema operativo) y otros gastos menores de material de oficina.

¹⁰En una reproducción profesional o a mayor escala podría utilizarse Google Colab Pro+ u otras plataformas de computación en la nube con GPU dedicada. Esta partida no se incluye en el presupuesto porque no se incurrió en ningún coste de infraestructura durante el desarrollo del trabajo.

Concepto	Total
Equipo personal (amortización parcial)	300,00 €
Google Colab (versión gratuita)	0,00 €
Software de código abierto	0,00 €
Costes indirectos (electricidad, Internet)	204,00 €
Subtotal infraestructura	204,00 €

TABLA 8.3. COSTE DE INFRAESTRUCTURA Y SOFTWARE
(ESTIMACIÓN)

8.5.3. Presupuesto total

La suma de ambos subtotales da el coste antes de impuestos. Sobre él se aplica el Impuesto sobre el Valor Añadido (IVA) del 21 %, como se haría en un presupuesto profesional hipotético. La Tabla 8.4 presenta el resultado final.

Concepto	Importe
Subtotal personal	9.319,50 €
Subtotal infraestructura	204,00 €
Subtotal	9.523,50 €
IVA (21 %)	1.999,94 €
Total	11.523,44 €

TABLA 8.4. PRESUPUESTO TOTAL (ESTIMACIÓN ACADÉMICA)

El importe total estimado del trabajo asciende a 11.523,44 €. Conviene insistir en que se trata de una estimación académica orientativa, no de un coste realmente facturado, y que la partida más relevante es, con diferencia, el tiempo de dedicación del estudiante.

9. CONCLUSIONES Y TRABAJO FUTURO

El objetivo de este trabajo era evaluar y comparar distintas familias de modelos de Procesamiento del Lenguaje Natural para la clasificación binaria de frases de informes de malware como relevantes o no relevantes, según la definición de la SubTarea 1 de SemEval-2018 Task 8 [3]. Se han cubierto todos los objetivos planteados en el Capítulo 1: se preparó el dataset, se implementó un baseline reproducible con SVM y TF-IDF, se evaluaron modelos recurrentes BiLSTM, Transformers genéricos y de dominio, se exploró el aumento de datos por *back-translation* y la clasificación sin fine-tuning mediante *prompting*. El código y el dataset preprocesado se publicaron de forma abierta. El mejor resultado lo obtuvo la replicación de la arquitectura BiLSTM de Villani [5], con un F1 de 0,5541 sobre la clase relevante, por encima de todos los Transformers evaluados y a 1,7 puntos del sistema ganador de la competición oficial.

De ese conjunto de experimentos se ha llegado a una serie de conclusiones:

Uno de los resultados más claros del trabajo es que el tamaño del modelo no fue el factor que mejor explicó el rendimiento. La BiLSTM, con alrededor de 1,5 millones de parámetros entrenables, obtuvo mejores resultados que modelos Transformer mucho mayores (con 110 millones de parámetros), como BERT o RoBERTa. Los resultados sugieren que en presencia de un dataset pequeño y desbalanceado, la complejidad arquitectónica y el número de parámetros no son garantía de mejor rendimiento. La BiLSTM, con aproximadamente 1,5 millones de parámetros, superó a modelos como BERT o RoBERTa, con 110 millones. Una arquitectura más ligera, diseñada específicamente para las características del problema, puede ser más eficaz que un modelo de propósito general de mayor escala.

También se observa que el preentrenamiento en textos de ciberseguridad puede ser útil, pero no garantiza una mejora por sí solo. SecureBERT 2.0 [10] fue el Transformer con mejor rendimiento, con un F1 de 0,5515, mientras que CTI-BERT y SecBERT no superaron a BERT genérico. Esto indica que no basta con que un modelo esté etiquetado como modelo de dominio. El resultado parece depender también de la arquitectura base, del corpus utilizado en el preentrenamiento y de la actualización del propio modelo. Como trabajo futuro, sería interesante estudiar estos factores de forma más controlada, manteniendo constantes el resto de condiciones experimentales.

Otro aspecto relevante es la influencia del protocolo experimental. Las dos configuraciones de BERT difieren únicamente en el volumen de datos de entrenamiento (*train+dev* frente a *train*), manteniendo el resto de hiperparámetros idénticos. La diferencia de F1 resultante fue de menos de dos décimas (0,5164 frente a 0,5145), lo que indica que en este corpus el volumen adicional que aporta el conjunto de desarrollo tiene un impacto prácticamente nulo. Por ello, una comparación estricta entre arquitecturas requiere fijar de forma homogénea todas las condiciones del experimento. Una continuación natural sería repetir los entrenamientos con varias semillas aleatorias y reportar media y desviación

típica, aunque esto no fue viable en este trabajo por las restricciones de cómputo descritas en el Capítulo 7.

El aumento de datos mediante *back-translation* no produjo mejora apreciable en BERT: el F1 pasó de 0,5164 a 0,5145, una variación dentro del margen de ruido de una semilla aleatoria. Sin embargo, en el caso de SecureBERT no es posible aislar con precisión el efecto del aumento, porque la comparación disponible también introduce un cambio de modelo base. En futuros experimentos podría ampliarse esta estrategia con varios idiomas intermedios o combinarse con otras técnicas de aumento textual, siempre controlando que las nuevas frases mantengan el significado de las originales [52].

El *prompting*, en las condiciones evaluadas, quedó por debajo de los modelos ajustados. BART-MNLI obtuvo un F1 de 0,25 sobre una submuestra del conjunto de test y tendió a clasificar casi todas las frases como relevantes. Este resultado debe interpretarse con cautela, ya que el experimento estuvo condicionado por el hardware disponible. Modelos de lenguaje de mayor tamaño podrían comportarse de forma distinta, pero no podían ejecutarse localmente con los 16 GB de VRAM disponibles. Evaluar este enfoque mediante API externas o con una infraestructura más potente queda como una línea de trabajo futuro.

En conjunto, los resultados reflejan la dificultad de la tarea. Ningún modelo evaluado en este trabajo, ni tampoco los sistemas de referencia de la competición original, supera claramente el umbral de 0,58 en F1. Esta limitación puede explicarse por varios factores: el tamaño reducido del conjunto de datos, el desbalanceo entre clases, las particiones no estratificadas y la presencia de frases ambiguas en la frontera entre relevante y no relevante. En algunos casos, la etiqueta no resulta evidente incluso para un lector humano. Por ello, una revisión adicional de los casos frontera o un esquema de anotación con mayor nivel de detalle podrían ayudar a reducir parte del ruido del conjunto.

Más allá de la clasificación binaria, una ampliación natural del trabajo sería abordar las SubTareas 2, 3 y 4 de SemEval-2018 Task 8. Estas subtareas se centran en el etiquetado de los *tokens* relevantes, la predicción de relaciones entre ellos y la clasificación de acciones según la taxonomía MAEC (*Malware Attribute Enumeration and Characterization*) [3], [56]. Esto permitiría pasar de identificar frases relevantes a estructurar la información técnica que contienen.

Una línea de continuación más aplicada sería integrar el modelo con mejor equilibrio entre precisión y *recall* en el sistema de apoyo al analista descrito en el Capítulo 4. Ese capítulo recoge los requisitos funcionales y no funcionales de un sistema hipotético, desde la ingesta y segmentación de informes hasta el filtrado de frases relevantes y la exportación de resultados. En este trabajo dichos requisitos se formularon como referencia, pero no se implementó una herramienta integrada, ya que el alcance se limitó a la evaluación comparativa de modelos. Su desarrollo efectivo, respetando el marco regulador descrito en el Capítulo 8, queda como trabajo futuro. En cualquier caso, un sistema de este tipo debería plantearse como una herramienta de apoyo al analista, no como un sustituto de la

revisión experta.

BIBLIOGRAFÍA

- [1] M. Bayer, P. Kuehn y C. Reuter, “A Survey on Applications of Natural Language Processing for Cyber Security,” *ACM Computing Surveys*, vol. 56, n.º 6, pp. 1-38, 2024. doi: [10.1145/3638049](https://doi.org/10.1145/3638049).
- [2] N. Sun et al., “Cyber Threat Intelligence Mining for Proactive Cybersecurity Defense: A Survey and New Perspectives,” *IEEE Communications Surveys & Tutorials*, vol. 25, n.º 3, pp. 1748-1774, 2023. doi: [10.1109/COMST.2023.3273282](https://doi.org/10.1109/COMST.2023.3273282).
- [3] P. Phandi, A. Silva y W. Lu, “SemEval-2018 Task 8: Semantic Extraction from CybersecUrity REports using Natural Language Processing (SecureNLP),” en *Proceedings of the 12th International Workshop on Semantic Evaluation (SemEval)*, 2018, pp. 697-706. doi: [10.18653/v1/S18-1113](https://doi.org/10.18653/v1/S18-1113). [En línea]. Disponible en: <https://aclanthology.org/S18-1113/>.
- [4] S. K. Lim, A. O. Muis, W. Lu y C. H. Ong, “MalwareTextDB: A Database for Annotated Malware Articles,” en *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017, pp. 1557-1567. doi: [10.18653/v1/P17-1143](https://doi.org/10.18653/v1/P17-1143). [En línea]. Disponible en: <https://aclanthology.org/P17-1143/>.
- [5] P. Loyola, K. Gajananan, Y. Watanabe y F. Satoh, “Villani at SemEval-2018 Task 8: Semantic Extraction from Cybersecurity Reports using Representation Learning,” en *Proceedings of the 12th International Workshop on Semantic Evaluation (SemEval)*, 2018, pp. 885-889. doi: [10.18653/v1/S18-1143](https://doi.org/10.18653/v1/S18-1143). [En línea]. Disponible en: <https://aclanthology.org/S18-1143/>.
- [6] J. Devlin, M.-W. Chang, K. Lee y K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” en *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171-4186. doi: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423). [En línea]. Disponible en: <https://aclanthology.org/N19-1423/>.
- [7] P. He, X. Liu, J. Gao y W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” en *International Conference on Learning Representations (ICLR)*, 2021. [En línea]. Disponible en: <https://arxiv.org/abs/2006.03654>.
- [8] Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019. [En línea]. Disponible en: <https://arxiv.org/abs/1907.11692>.

- [9] Y. Park e Y. You, “A Pretrained Language Model for Cyber Threat Intelligence,” en *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2023, pp. 121-128. DOI: [10.18653/v1/2023.emnlp-industry.12](https://doi.org/10.18653/v1/2023.emnlp-industry.12). [En línea]. Disponible en: <https://aclanthology.org/2023.emnlp-industry.12/>.
- [10] E. Aghaei et al., “SecureBERT 2.0: Advanced Language Model for Cybersecurity Intelligence,” *arXiv preprint arXiv:2510.00240*, 2025. [En línea]. Disponible en: <https://arxiv.org/abs/2510.00240>.
- [11] Jackaduma, *SecBERT: A Pretrained Language Model for Cyber Security Text*, HuggingFace, 2020. [En línea]. Disponible en: <https://huggingface.co/jackaduma/SecBERT>.
- [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011. [En línea]. Disponible en: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [13] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. [En línea]. Disponible en: <https://arxiv.org/abs/1912.01703>.
- [14] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi et al., “Transformers: State-of-the-Art Natural Language Processing,” en *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38-45. DOI: [10.18653/v1/2020.emnlp-demos.6](https://doi.org/10.18653/v1/2020.emnlp-demos.6). [En línea]. Disponible en: <https://aclanthology.org/2020.emnlp-demos.6/>.
- [15] D. Jurafsky y J. H. Martin, *Speech and Language Processing*, 3rd. Draft, 2024, Disponible en: <https://web.stanford.edu/~jurafsky/slp3/>.
- [16] Y. Goldberg, “A Primer on Neural Network Models for Natural Language Processing,” *Journal of Artificial Intelligence Research*, vol. 57, pp. 345-420, 2016. DOI: [10.1613/jair.4992](https://doi.org/10.1613/jair.4992). [En línea]. Disponible en: <https://jair.org/index.php/jair/article/view/11030>.
- [17] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado y J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013. [En línea]. Disponible en: <https://arxiv.org/abs/1310.4546>.
- [18] J. Pennington, R. Socher y C. D. Manning, “GloVe: Global Vectors for Word Representation,” en *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532-1543. [En línea]. Disponible en: <https://aclanthology.org/D14-1162/>.
- [19] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.

- [20] C. Cortes y V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, n.º 3, pp. 273-297, 1995. doi: [10.1007/BF00994018](https://doi.org/10.1007/BF00994018).
- [21] M. E. Maron y J. L. Kuhns, "Automatic Indexing: An Experimental Inquiry," *Journal of the ACM*, vol. 8, n.º 3, pp. 404-417, 1961. doi: [10.1145/321075.321084](https://doi.org/10.1145/321075.321084).
- [22] K. Kowsari, K. Jafari Meimandi, M. Heidarysafa, S. Mendu, L. E. Barnes y D. E. Brown, "Text Classification Algorithms: A Survey," *Information*, vol. 10, n.º 4, p. 150, 2019. doi: [10.3390/info10040150](https://doi.org/10.3390/info10040150).
- [23] J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua y A. Lopez, "A comprehensive survey on support vector machine classification: Applications, challenges and trends," *Neurocomputing*, vol. 408, pp. 189-215, 2020. doi: [10.1016/j.neucom.2019.10.118](https://doi.org/10.1016/j.neucom.2019.10.118).
- [24] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," en *Proceedings of the 10th European Conference on Machine Learning (ECML)*, 1998, pp. 137-142. doi: [10.1007/BFb0026683](https://doi.org/10.1007/BFb0026683).
- [25] T. M. Cover, "Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition," *IEEE Transactions on Electronic Computers*, vol. EC-14, n.º 3, pp. 326-334, 1965. doi: [10.1109/PGEC.1965.264137](https://doi.org/10.1109/PGEC.1965.264137).
- [26] H. Zhang, "The Optimality of Naive Bayes," en *Proceedings of the Seventeenth International Florida Artificial Intelligence Research Society Conference (FLAIRS)*, 2004, pp. 562-567. [En línea]. Disponible en: <https://www.cs.unb.ca/~hzhang/publications/FLAIRS04ZhangH.pdf>.
- [27] D. W. Otter, J. R. Medina y J. K. Kalita, "A Survey of the Usages of Deep Learning for Natural Language Processing," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, n.º 2, pp. 604-624, 2021. doi: [10.1109/TNNLS.2020.2979670](https://doi.org/10.1109/TNNLS.2020.2979670).
- [28] J. L. Elman, "Finding Structure in Time," *Cognitive Science*, vol. 14, n.º 2, pp. 179-211, 1990. doi: [10.1207/s15516709cog1402_1](https://doi.org/10.1207/s15516709cog1402_1).
- [29] Y. Kim, "Convolutional Neural Networks for Sentence Classification," en *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746-1751. doi: [10.3115/v1/D14-1181](https://doi.org/10.3115/v1/D14-1181). [En línea]. Disponible en: <https://aclanthology.org/D14-1181/>.
- [30] S. Hochreiter y J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, n.º 8, pp. 1735-1780, 1997. doi: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [31] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," en *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724-1734. doi: [10.3115/v1/D14-1179](https://doi.org/10.3115/v1/D14-1179). [En línea]. Disponible en: <https://aclanthology.org/D14-1179/>.

- [32] Y. Yu, X. Si, C. Hu y J. Zhang, “A Review of Recurrent Neural Networks: LSTM Cells and Network Architectures,” *Neural Computation*, vol. 31, n.º 7, pp. 1235-1270, 2019. doi: [10.1162/neco_a_01199](https://doi.org/10.1162/neco_a_01199).
- [33] A. Vaswani et al., “Attention Is All You Need,” en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [En línea]. Disponible en: <https://arxiv.org/abs/1706.03762>.
- [34] T. Lin, Y. Wang, X. Liu y X. Qiu, “A Survey of Transformers,” *AI Open*, vol. 3, pp. 111-132, 2022. doi: [10.1016/j.aiopen.2022.10.001](https://doi.org/10.1016/j.aiopen.2022.10.001).
- [35] C. Sun, X. Qiu, Y. Xu y X. Huang, “How to Fine-Tune BERT for Text Classification?” En *Proceedings of the 18th China National Conference on Chinese Computational Linguistics (CCL 2019)*, ép. Lecture Notes in Computer Science, vol. 11856, Springer, 2019, pp. 194-206. doi: [10.1007/978-3-030-32381-3_16](https://doi.org/10.1007/978-3-030-32381-3_16).
- [36] A. Radford, K. Narasimhan, T. Salimans e I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI, inf. téc., 2018. [En línea]. Disponible en: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [37] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023. [En línea]. Disponible en: <https://arxiv.org/abs/2302.13971>.
- [38] C. Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” *Journal of Machine Learning Research*, vol. 21, n.º 140, pp. 1-67, 2020. [En línea]. Disponible en: <https://jmlr.org/papers/v21/20-1307.html>.
- [39] M. Lewis et al., “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” en *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 7871-7880. doi: [10.18653/v1/2020.acl-main.703](https://doi.org/10.18653/v1/2020.acl-main.703). [En línea]. Disponible en: <https://aclanthology.org/2020.acl-main.703/>.
- [40] Y. Tay, M. Dehghani, D. Bahri y D. Metzler, “Efficient Transformers: A Survey,” *ACM Computing Surveys*, vol. 55, n.º 6, pp. 1-28, 2022. doi: [10.1145/3530811](https://doi.org/10.1145/3530811).
- [41] I. Beltagy, M. E. Peters y A. Cohan, “Longformer: The Long-Document Transformer,” en *arXiv preprint arXiv:2004.05150*, 2020. [En línea]. Disponible en: <https://arxiv.org/abs/2004.05150>.
- [42] P. Ranade, A. Piplai, S. Mittal, A. Joshi y T. Finin, “CyBERT: Contextualized Embeddings for the Cybersecurity Domain,” en *IEEE International Conference on Big Data*, 2021, pp. 3334-3342. doi: [10.1109/BigData52589.2021.9671693](https://doi.org/10.1109/BigData52589.2021.9671693).

- [43] E. Aghaei, X. Niu, W. Shadber y E. Al-Shaer, “SecureBERT: A Domain-Specific Language Model for Cybersecurity,” en *Security and Privacy in Communication Networks (SecureComm)*, Springer, 2023, pp. 39-56. doi: [10.1007/978-3-031-25538-0_3](https://doi.org/10.1007/978-3-031-25538-0_3).
- [44] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou et al., “A Survey of Large Language Models,” *arXiv preprint arXiv:2303.18223*, 2023. [En línea]. Disponible en: <https://arxiv.org/abs/2303.18223>.
- [45] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever, “Language Models are Unsupervised Multitask Learners,” OpenAI, inf. téc., 2019. [En línea]. Disponible en: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [46] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877-1901, 2020. [En línea]. Disponible en: <https://arxiv.org/abs/2005.14165>.
- [47] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023. [En línea]. Disponible en: <https://arxiv.org/abs/2307.09288>.
- [48] OpenAI, *Introducing GPT-5*, <https://openai.com/es-ES/index/introducing-gpt-5/>, Accedido: 10 de junio de 2026, 2025.
- [49] OpenAI, *Introducing GPT-5.5*, <https://openai.com/index/introducing-gpt-5-5/>, Accedido: 10 de junio de 2026, 2026.
- [50] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus et al., “Scaling Instruction-Finetuned Language Models,” *arXiv preprint arXiv:2210.11416*, 2022. [En línea]. Disponible en: <https://arxiv.org/abs/2210.11416>.
- [51] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi y G. Neubig, “Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing,” *ACM Computing Surveys*, vol. 55, n.º 9, pp. 1-35, 2023. doi: [10.1145/3560815](https://doi.org/10.1145/3560815).
- [52] S. Y. Feng et al., “A Survey of Data Augmentation Approaches for NLP,” *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 968-988, 2021. doi: [10.18653/v1/2021.findings-acl.84](https://doi.org/10.18653/v1/2021.findings-acl.84).
- [53] J. Wei y K. Zou, “EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks,” en *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 6382-6388. doi: [10.18653/v1/D19-1670](https://doi.org/10.18653/v1/D19-1670). [En línea]. Disponible en: <https://aclanthology.org/D19-1670/>.
- [54] E. Ma, *NLP Augmentation*, GitHub, Disponible en: <https://github.com/makcedward/nlpaug>, 2019.

- [55] R. Sennrich, B. Haddow y A. Birch, “Improving Neural Machine Translation Models with Monolingual Data,” en *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 86-96. doi: [10.18653/v1/P16-1009](https://doi.org/10.18653/v1/P16-1009). [En línea]. Disponible en: <https://aclanthology.org/P16-1009/>.
- [56] I. Kirillov, D. Beck, P. Chase y R. Martin, “Malware Attribute Enumeration and Characterization,” The MITRE Corporation, inf. téc., 2010.
- [57] U. K. Sikdar, B. Barik y B. Gambäck, “Flytxt_NTNU at SemEval-2018 Task 8: Identifying and Classifying Malware Text Using Conditional Random Fields and Naïve Bayes Classifiers,” en *Proceedings of the 12th International Workshop on Semantic Evaluation (SemEval)*, 2018, pp. 890-893. doi: [10.18653/v1/S18-1144](https://doi.org/10.18653/v1/S18-1144). [En línea]. Disponible en: <https://aclanthology.org/S18-1144/>.
- [58] M. Fu, X. Zhao e Y. Yan, “HCCL at SemEval-2018 Task 8: An End-to-End System for Sequence Labeling from Cybersecurity Reports,” en *Proceedings of the 12th International Workshop on Semantic Evaluation (SemEval)*, 2018, pp. 874-877. doi: [10.18653/v1/S18-1141](https://doi.org/10.18653/v1/S18-1141). [En línea]. Disponible en: <https://aclanthology.org/S18-1141/>.
- [59] X. Ma y E. Hovy, “End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF,” en *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1064-1074. doi: [10.18653/v1/P16-1101](https://doi.org/10.18653/v1/P16-1101). [En línea]. Disponible en: <https://aclanthology.org/P16-1101/>.
- [60] M. Ravikiran, K. Madgula y S. Saha, “TeamDL at SemEval-2018 Task 8: Cybersecurity Text Analysis using Convolutional Neural Network and Conditional Random Fields,” en *Proceedings of the 12th International Workshop on Semantic Evaluation (SemEval)*, 2018, pp. 868-873. doi: [10.18653/v1/S18-1140](https://doi.org/10.18653/v1/S18-1140). [En línea]. Disponible en: <https://aclanthology.org/S18-1140/>.
- [61] A. Padia et al., “UMBC at SemEval-2018 Task 8: Understanding Text about Malware,” en *Proceedings of the 12th International Workshop on Semantic Evaluation (SemEval)*, 2018, pp. 878-884. doi: [10.18653/v1/S18-1142](https://doi.org/10.18653/v1/S18-1142). [En línea]. Disponible en: <https://aclanthology.org/S18-1142/>.
- [62] C. Brew, “Digital Operatives at SemEval-2018 Task 8: Using Dependency Features for Malware NLP,” en *Proceedings of the 12th International Workshop on Semantic Evaluation (SemEval)*, 2018, pp. 894-897. doi: [10.18653/v1/S18-1145](https://doi.org/10.18653/v1/S18-1145). [En línea]. Disponible en: <https://aclanthology.org/S18-1145/>.
- [63] K. Ameri, M. Hempel, H. Sharif, J. Lopez Jr. y K. Perumalla, “CyBERT: Cybersecurity Claim Classification by Fine-Tuning the BERT Language Model,” *Journal of Cybersecurity and Privacy*, vol. 1, n.º 4, pp. 615-637, 2021. doi: [10.3390/jcp1040031](https://doi.org/10.3390/jcp1040031).

- [64] D. Bahdanau, K. Cho e Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” *arXiv preprint arXiv:1409.0473*, 2015. [En línea]. Disponible en: <https://arxiv.org/abs/1409.0473>.
- [65] A. Williams, N. Nangia y S. Bowman, “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference,” en *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2018, pp. 1112-1122. doi: [10.18653/v1/N18-1101](https://doi.org/10.18653/v1/N18-1101). [En línea]. Disponible en: <https://aclanthology.org/N18-1101/>.
- [66] J. Tiedemann, “Parallel Data, Tools and Interfaces in OPUS,” en *Proceedings of the Eight International Conference on Language Resources and Evaluation (LREC)*, 2012, pp. 2214-2218. [En línea]. Disponible en: <https://aclanthology.org/L12-1246/>.
- [67] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, n.º 13, pp. 3521-3526, 2017. doi: [10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114).
- [68] Parlamento Europeo y Consejo de la Unión Europea, *Directiva (UE) 2022/2555 del Parlamento Europeo y del Consejo relativa a las medidas destinadas a garantizar un elevado nivel común de ciberseguridad en toda la Unión (NIS2)*, Diario Oficial de la Unión Europea, Entró en vigor el 16 de enero de 2023. [En línea]. Disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32022L2555>, 2022.
- [69] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (Reglamento General de Protección de Datos)*, Diario Oficial de la Unión Europea, Versión consolidada que incorpora las correcciones publicadas en DOUE L 127 de 23.5.2018 y DOUE L 74 de 4.3.2021. [En línea]. Disponible en: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016.
- [70] España, *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, Boletín Oficial del Estado, BOE núm. 294. Modificada por la Ley 11/2023, de 8 de mayo (BOE núm. 111). [En línea]. Disponible en: <https://www.boe.es/eli/es/lo/2018/12/05/3/con>, 2018.
- [71] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo por el que se establecen normas armonizadas en materia de inteligencia artificial (Ley de Inteligencia Artificial)*, Diario Oficial de la Unión Europea, [En línea]. Disponible en: https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=OJ:L_202401689, 2024.

- [72] Instituto Nacional de Ciberseguridad (INCIBE) y Confederación Española de Empresas de Tecnologías de la Información, Comunicaciones y Electrónica (CONETIC), *Estudio sobre la industria de la ciberseguridad en España 2025*, [En línea], Presentado el 18 de marzo de 2026. Disponible en: <https://www.incibe.es/incibe/sala-de-prensa/incibe-y-conetic-presentan-las-conclusiones-del-estudio-sobre-la-industria-de>, 2026.
- [73] Observatorio Sectorial DBK de Informa, *Ciberseguridad en España: el mercado alcanza los 2.500 millones de euros en 2024 con un crecimiento del 14,2 %*, [En línea], Disponible en: <https://www.audidat.com/blog/noticias/crecimiento-ciberseguridad-espana-2024-2026/>, 2025.
- [74] Programa Nacional de Algoritmos Verdes (PNAV), *Reducción de la huella de carbono en modelos de IA: un paso hacia una IA más sostenible*, [En línea], Disponible en: <https://algoritmosverdes.gob.es/es/noticias/reduccion-de-la-huella-de-carbono-en-modelos-de-ia-un-paso-hacia-una-ia-mas-sostenible>, 2024.
- [75] Naciones Unidas, *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible*, Resolución aprobada por la Asamblea General el 25 de septiembre de 2015, [En línea]. Disponible en: <https://www.un.org/sustainabledevelopment/es/sustainable-development-goals/>, 2015.
- [76] UNESCO, *La inteligencia artificial en la educación*, [En línea], Disponible en: <https://www.unesco.org/es/digital-education/artificial-intelligence>, 2023.
- [77] E. Medina y J. Gutiérrez, “Pautas para optar por una metodología ágil para proyectos de software,” *Revista Educación en Ingeniería*, vol. 19, n.º 37, pp. 1-8, 2024. DOI: [10.26507/rei.v19n37.1292](https://doi.org/10.26507/rei.v19n37.1292). [En línea]. Disponible en: <https://educacioneningenieria.org/index.php/edi/article/view/1292>.
- [78] A. P. Largo-Anguisaca y D. X. Poma-Japón, “Comparación de metodologías ágiles para el desarrollo de software,” *MQRInvestigar*, vol. 8, n.º 1, pp. 5052-5074, 2024. DOI: [10.56048/MQR20225.8.1.2024.5052-5074](https://doi.org/10.56048/MQR20225.8.1.2024.5052-5074).
- [79] España, *Real Decreto 1125/2003, de 5 de septiembre, por el que se establece el sistema europeo de créditos y el sistema de calificaciones en las titulaciones universitarias de carácter oficial y validez en todo el territorio nacional*, Boletín Oficial del Estado, BOE núm. 224. [En línea]. Disponible en: <https://www.boe.es/eli/es/rd/2003/09/05/1125>, 2003.
- [80] Instituto Nacional de Estadística, *Encuesta Anual de Estructura Salarial (EAES). Año 2023. Datos definitivos*, [En línea], Publicado el 28 de mayo de 2025. Disponible en: <https://www.ine.es/dyngs/Prensa/EAES2023.htm>, 2025.

- [81] España, *Orden PJC/178/2025, de 25 de febrero, por la que se desarrollan las normas legales de cotización a la Seguridad Social, desempleo, protección por cese de actividad, Fondo de Garantía Salarial y formación profesional para el ejercicio 2025*, Boletín Oficial del Estado, BOE-A-2025-3780. [En línea]. Disponible en: https://www.boe.es/diario_boe/txt.php?id=BOE-A-2025-3780, 2025.
- [82] CostEEnergía, *Histórico del precio PVPC anual de la electricidad en España*, [En línea], Disponible en: <https://www.costeenergia.es/historico/pvpc-anual>, 2025.
- [83] Selectra España, *Ofertas de Internet: precios y características*, [En línea], Accedido en mayo de 2026. Disponible en: <https://selectra.es/internet-telefono/internet>, 2026.

ANEXO A: DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA (IAG) EN EL TRABAJO DE FIN DE GRADO (TFG)

He usado IAG en mi TFG

Marca lo que corresponda:

<u>SÍ</u> ✓	NO
-------------	----

Si has marcado SÍ, completa las siguientes 3 partes de este documento:

Parte 1: declaración sobre comportamiento legal, ético y responsable

Ten presente que el uso de IAG conlleva unos riesgos y puede generar una serie de consecuencias académicas graves: la evaluación de tu TFG puede verse comprometida si el uso de la IAG comporta la utilización de datos de carácter confidencial, materiales protegidos por derechos de autoría, o datos de carácter personal, y se hace sin cumplir las condiciones exigidas en cada caso (autorización de los interesados, autorización de los titulares, seguimiento de las instrucciones de la Universidad).

Pregunta	
En mi interacción con herramientas de IAG he facilitado datos de carácter confidencial contando siempre con la debida autorización de los interesados. La confidencialidad abarca toda información que una persona u organización desea proteger por razones legales, comerciales, de privacidad o estratégicas (como patentes o secretos comerciales).	
SÍ, he usado estos datos con la autorización de los interesados	NO, no he usado datos de carácter confidencial
En mi interacción con herramientas de IAG he facilitado materiales protegidos por derechos de autoría contando siempre con la autorización de los respectivos titulares.	

SÍ, he usado estos materiales con autorización de los titulares de derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley: obra en dominio público, obra licenciada (licencias Creative Commons), uso de fragmentos con fines de investigación (derecho de cita)	NO, no he usado materiales protegidos por derechos de autoría
--	--

En mi interacción con herramientas de IAG he facilitado datos de carácter personal con la debida autorización de los interesados.
--

SÍ, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad	NO, no he usado datos de carácter personal
---	---

Mi utilización de la herramienta de IAG ha respetado sus términos de uso , así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.

SÍ	NO
-----------	-----------

Parte 2: declaración de uso técnico

Utiliza el siguiente modelo de declaración tantas veces como sea necesario, a fin de reflejar todos los tipos de iteración que has tenido con herramientas de IAG. Incluye un ejemplo por cada tipo de uso realizado donde se indique: *[Añade un ejemplo]*.

Declaro haber hecho uso del sistema de IAG (*ChatGPT* y *Claude*) para:

Documentación y redacción:

- *Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IAG*

He consultado distintas formas de estructurar el análisis comparativo de familias de modelos de PLN (SVM, BiLSTM, Transformers) para la tarea de clasificación binaria de frases de informes de malware.

- *Revisión o reescritura de párrafos redactados previamente*

He solicitado la reescritura de párrafos de los capítulos de introducción, fundamentos teóricos y conclusiones para mejorar la cohesión, reducir la longitud y mantener las mismas ideas

- *Búsqueda de información o respuesta a preguntas concretas*

He realizado preguntas orientadas a encontrar definiciones técnicas (mecanismo de atención, back-translation, métricas de evaluación en clasificación desequilibrada) y aclaraciones sobre el marco regulador aplicable (RGPD, LOPDGDD)

- *Búsqueda de bibliografía*

Además de buscar referencias por mi cuenta, he consultado como herramienta adicional la IAG para buscar papers de modelos de PLN aplicados a ciberseguridad como SecureBERT.

- *Resumen de bibliografía consultada*

He pedido resúmenes de papers técnicos para orientarme sobre su relevancia antes de leerlos en detalle, en particular trabajos sobre fine-tuning de BERT y modelos especializados en ciberseguridad.

- *Traducción de textos consultados*

He solicitado la traducción al español de fragmentos de papers en inglés para facilitar su comprensión, verificando siempre el sentido técnico del original.

Desarrollar contenido específico

Se ha hecho uso de IAG como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- *Asistencia en el desarrollo de líneas de código (programación)*

He pedido sugerencias sobre librerías para preparar el dataset, el entrenamiento o evaluación de modelos, así como para depurar fragmentos de código Python.

- *Generación de esquemas, imágenes, audios o vídeos*

He utilizado Claude (Anthropic) para generar figuras explicativas incluidas en la memoria: la distribución de clases por partición, el diagrama del mecanismo de auto-atención y el diagrama del hiperplano de máximo margen en SVM. Todas las figuras se generaron mediante scripts Python con matplotlib y se etiquetan como elaboración propia en su respectivos pies de figura.

- *Procesos de optimización*

He solicitado ayuda para interpretar mensajes de error durante el entrenamiento de modelos en entornos con memoria GPU limitada (Colab gratuito, T4/P100) y para ajustar hiperparámetros de la búsqueda en cuadrícula del SVM.

- *Tratamiento de datos: recogida, análisis, cruce de datos...*

He pedido orientación sobre estrategias de aumento de datos mediante back-translation (ruta inglés-francés-inglés) y sobre cómo fusionar correctamente los subconjuntos de entrenamiento y validación del corpus SemEval-2018 Task 8.

Si crees necesario incluirlo, añade, en cada uno de los casos:

empleando el *prompt*...

(escribe la petición realizada a la IAG)

teniendo como interacción...

(describe qué interacción realizaste con la IAG tras la respuesta del prompt)

*Ejemplo: **Declaro** haber hecho uso del sistema de IAG **Chat GPT X.x** para **buscar información** empleando **el prompt**: “Dime un ejemplo concreto que ilustre la aplicación de la propuesta urbanística de la Ciudad de los 15 minutos en España” **teniendo como interacción** la proposición del ejemplo concreto del distrito de Chamartín que se ha reflejado en la memoria.*

Parte 3: reflexión sobre utilidad

Aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IAG en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

El uso de herramientas de inteligencia artificial generativa (IAG) ha servido como apoyo durante el desarrollo del trabajo, sobre todo en tareas técnicas y repetitivas. Se utilizaron, por ejemplo, para revisar errores de código Python, preparar figuras con *matplotlib*, organizar tablas en LaTeX y contrastar posibles formas de estructurar algunos apartados de la memoria.

Su utilidad principal ha estado en acelerar tareas mecánicas, no en sustituir el trabajo de análisis. Las decisiones importantes del proyecto, como la elección de los modelos, la interpretación de los resultados o la formulación de las conclusiones, se tomaron a partir del estudio realizado y de los experimentos ejecutados.

También se detectaron limitaciones claras. El código generado no siempre funcionaba correctamente y, en algunos casos, las respuestas no se ajustaban exactamente a lo solicitado. Por ello, fue necesario revisar, probar y corregir las salidas antes de incorporarlas al trabajo. Lo mismo ocurrió con las sugerencias bibliográficas, que se verificaron antes de utilizarse en la memoria.

En conjunto, la IAG ha sido una herramienta útil para apoyar el proceso de desarrollo y redacción, especialmente en tareas auxiliares. Sin embargo, su uso requirió supervisión constante y no sustituyó el razonamiento crítico necesario para elaborar un trabajo académico de estas características.